Importing the Necessary libraries in Google Colab using python

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Figure 1 - Code to import necessary libraries

This code snippet establishes the groundwork for data analysis and visualization. The aliases (**pd**, **np**, **plt**, **sns**) make it more convenient to use these libraries throughout the code. Once imported, these libraries enable various data-related operations, numerical computations, and the creation of visualizations to facilitate effective analysis of data. The explanation of the code is provided below.

Pandas (import pandas as pd):

- “Pandas” is a powerful library in Python for data manipulation and analysis.
- The import pandas as pd statement aliases the library name to 'pd,' making it easier to reference in the code.

NumPy (import numpy as np):

- “Numpy” is a library that supports large, multi-dimensional arrays and matrices, along with mathematical functions to operate on these structures.
- The import numpy as np statement aliases the library name to 'np' for simplicity.

Matplotlib (import matplotlib.pyplot as plt):

- “matplotlib.pyplot” is a module within the matplotlib library, widely used for creating static, animated, and interactive visualizations in Python.
- The import matplotlib.pyplot as plt statement aliases the module name to 'plt' for brevity in the code.

Seaborn (import seaborn as sns):

- “seaborn” is built on top of matplotlib and provides a high-level interface for creating attractive statistical graphics.
- The import seaborn as sns statement aliases the library name to 'sns' for easier referencing.

Importing a Specific Module and uploading Files

```
from google.colab import files
uploaded = files.upload()
```

Choose Files 2 files

- **Customers.csv**(text/csv) - 220188 bytes, last modified: 11/19/2023 - 100% done
- **Sales.csv**(text/csv) - 19294 bytes, last modified: 11/28/2023 - 100% done

```
Saving Customers.csv to Customers.csv
Saving Sales.csv to Sales.csv
```

Figure 2 - Code for Importing a Specific Module and uploading Files

These two lines of code enable users in Google Colab to upload files from their local devices. The **files** module provides a convenient interface for handling file operations within the Colab environment, facilitating the seamless integration of external data into the Python code being executed in a Colab notebook.

Importing a Specific Module - “google.colab import files”

- The from google.colab import files statement is used to import a specific module named files from the google.colab package.
- In Google Colab, the files module provides functionality for interacting with files, particularly for uploading and downloading files within the Colab environment.

Uploading Files “Uploaded = files.upload()”

- The files.upload() function is called to initiate the file uploading process in Google Colab.
- When this line of code is executed, it prompts the user to select files from their local machine for uploading.
- The uploaded files are then stored in the uploaded variable, which typically holds a dictionary where keys are the file names, and values are the content of the files.

Reading CSV Files

```
[5] import io
    df_customers = pd.read_csv(io.BytesIO(uploaded['Customers.csv']))
    df_sales = pd.read_csv(io.BytesIO(uploaded['Sales.csv']))
```

Figure 3 - Code to read CSV files

This part of the code focuses on reading CSV files that have been uploaded to Google Colab. By using Pandas and the io module, the code efficiently handles the content of these files, converting them into DataFrame objects (df_customers and df_sales), which are then available for further data analysis and manipulation in the Colab environment.

Importing the io Module (import io):

- The import io statement brings in the io module, which stands for "input/output." This module provides tools for working with various types of input and output, including in-memory file-like objects.

Reading CSV Files Using Pandas “df_customers = pd.read_csv (io.BytesIO (uploaded ['Customers.csv']))”

- The pd.read_csv() function from the Pandas library is employed to read CSV (Comma-Separated Values) files.
- io.BytesIO() is used to create an in-memory binary stream from the uploaded content. This is necessary because Pandas' read_csv function expects a file-like object.
- uploaded['Customers.csv'] retrieves the content of the 'Customers.csv' file from the dictionary obtained during the file upload process.
- The resulting DataFrame (df_customers) contains the structured data from the 'Customers.csv' file, making it accessible for further analysis.

Reading Another CSV File (df_sales = pd.read_csv (io.BytesIO(uploaded['Sales.csv'])))

- Similarly, the code reads another CSV file, 'Sales.csv,' using the same approach.

- The content of 'Sales.csv' is obtained from uploaded['Sales.csv'], and an in-memory binary stream is created using io.BytesIO().
- The resulting DataFrame (df_sales) holds the structured data from the 'Sales.csv' file.

Displaying the first few rows of each dataset

```python
# Display the first few rows of each dataset
print("Customers Data:")
print(df_customers.head())

print("\nSales Data:")
print(df_sales.head())
```

```
omers Data:
,tYear_Birth\tEducation\tMarital_Status\tIncome\tKidhome\tTeenhome\tDt_Customer\tRecency\tMntWines\tMntFr
;24\t1957\tGraduation\tSingle\t58138\t0\t0\t0...
.74\t1954\tGraduation\tSingle\t46344\t1\t1\t0...
.41\t1965\tGraduation\tTogether\t71613\t0\t0\...
.82\t1984\tGraduation\tTogether\t26646\t1\t0\...
;24\t1981\tPhD\tMarried\t58293\t1\t0\t19-01-2...

; Data:
:egory 1   Quality        Country     1994     1995     1996     1997  \
    Wine       All         Total   1,535.2  1,581.9  1,606.7  1,701.0
    Wine  Standard         Total   1,404.8  1,438.8  1,461.8  1,546.9
    Wine  Standard  United States    243.0    214.0    219.5    225.0
    Wine  Standard        Ireland    537.5    546.0    513.0    524.5
    Wine  Standard         France     92.0    118.0    137.0    151.0

  1998     1999     2000  ...     2011     2012     2013     2014  \
 839.4  1,933.5  2,124.1  ...  3,860.4  4,235.1  4,237.6  4,735.4
 667.1  1,752.7  1,917.1  ...  3,618.3  3,970.0  3,996.0  4,449.6
 239.5    249.0    259.0  ...    803.0    975.5  1,099.3  1,337.6
 526.0    517.5    536.4  ...    473.4    448.0    404.4    404.3
 161.0    181.0    225.0  ...    370.3    370.0    353.0    380.0

  2015     2016     2017     2018     2019     2020
 447.0  6,022.3  6,622.7  6,931.2  7,425.1  8,250.8
 125.6  5,676.8  6,247.6  6,500.0  6,888.2  7,536.6
 652.3  1,989.8  2,365.0  2,463.8  2,893.3  3,337.5
 426.1    484.5    468.8    413.9    434.6    444.2
 376.0    365.3    364.0    364.0    358.3    345.3

ows x 30 columns]
```

Figure 4 - Code to display first few rows of both datasets

Printing Header Information (print("Customers Data:") and print("Sales Data:")):

- These lines of code use the print() function to display informative headers before presenting the data. This helps in clearly identifying which dataset is being shown.

Displaying the First Few Rows of the Datasets (print(df_customers.head()) and print(df_sales.head())):

- df_customers.head() and df_sales.head() are calls to the head() method of Pandas DataFrames (df_customers and df_sales, respectively).
- The head () method retrieves the first few rows of the DataFrame. By default, it returns the first five rows, giving a snapshot of the dataset.
- The displayed information includes the column names and the values in the initial rows, offering a quick overview of the dataset's structure and content.

Exploring the Data

```
print("Customer Data Summary:")
print(customer_data.describe())

print("\nSales Data Summary:")
print(sales_data.describe())
```

```
Customer Data Summary:
       ID\tYear_Birth\tEducation\tMarital_Status\tIncome\tKidhome\tTeenhome\tDt_Customer\tRecency\tMntWines\tMntFruits\tMntMeatProducts\tMntFishProducts\tMntSw
count                                                   2240
unique                                                  2240
top     5524\t1957\tGraduation\tSingle\t58138\t0\t0\t0...
freq                                                       1

Sales Data Summary:
       Category 1   Quality Country 1994 1995 1996 1997 1998 1999 2000  ...  \
count         157       157     157   62   62   66   70   90   98   95  ...
unique          1         4      77   41   44   43   47   55   58   58  ...
top          Wine  Standard  Total   0.0  0.0  0.0  0.0  0.1  0.1  0.0  ...
freq          157        52      4   13   13   12   11   12   11   12  ...

       2011 2012 2013 2014 2015 2016 2017 2018 2019 2020
count   117  120  119  117  130  131  134  132  132  134
unique   88   85   87   90   93   99   96   94   97   96
top     0.1  0.0  0.1  0.1  0.0  0.0  0.0  0.0  0.1  0.0
freq      7    7    7    6   10   10   11    9    8    9

[4 rows x 30 columns]
```

Figure 5 - exploring data

This code provides a quick overview of the statistical properties of the numeric columns in the "Customer Data" and "Sales Data" datasets. It is a helpful step in understanding the characteristics and distributions of the data before delving into more detailed analysis or visualization.

Printing Summary Information

- The print("Customer Data Summary:") statement serves as a header to indicate that the following information pertains to the "Customer Data" dataset.
- print(customer_data.describe()) prints descriptive statistics of the "Customer Data" DataFrame using the describe() function.
- Similarly, the print("Sales Data Summary:") statement acts as a header for the "Sales Data" dataset.
- print(sales_data.describe()) prints the summary statistics of the "Sales Data" DataFrame using the describe() function.

Describe() Function

- The describe() function in Pandas is used to generate descriptive statistics of a DataFrame.
- For numeric data, it provides statistics such as the count, mean, standard deviation, minimum, 25th percentile (Q1), median (50th percentile or Q2), 75th percentile (Q3), and maximum.
- These statistics offer insights into the central tendency, dispersion, and shape of the distribution of the data.

Interpreting the Output

- The output includes a table with various statistical measures for each numeric column in the datasets.
- The "count" row indicates the number of non-missing values in each column.
- The "mean" represents the average value, while "std" is the standard deviation, a measure of the spread of the values.
- The quartiles (Q1, Q2, Q3) provide information about the distribution of the data.

Cleaning and pre-processing the data

```
[27] customer_data.dropna(inplace=True)
     sales_data.fillna(0, inplace=True)  # Fill missing values with 0 in the sales data
```

Figure 6 - Dtaa cleaning and preprocessing

This code segment is responsible for cleaning the data by removing rows with missing values in "Customer Data" and filling missing values with zeros in "Sales Data." These steps contribute to ensuring the datasets are prepared for subsequent analysis without being adversely affected by missing data.

Handling Missing Values in "Customer Data":

- customer_data.dropna(inplace=True) is a Pandas DataFrame operation that removes rows containing missing (NaN) values.
- The dropna() method is applied to the "Customer Data" DataFrame, and the inplace=True parameter ensures that the changes are made directly to the original DataFrame without the need for assignment.

Filling Missing Values in "Sales Data":

- sales_data.fillna(0, inplace=True) is a Pandas DataFrame operation that fills missing values with a specified value, in this case, 0.
- The fillna() method is applied to the "Sales Data" DataFrame, and the inplace=True parameter ensures that the changes are made directly to the original DataFrame without the need for assignment.

Purpose of Handling Missing Values:

- Missing values in a dataset can impact the accuracy of analyses and visualizations. Therefore, it is common practice to handle missing data before proceeding with further analysis.
- In the case of dropping missing values in "Customer Data," this approach may be suitable if the number of missing values is relatively small, and removing those rows does not significantly affect the overall dataset.
- For "Sales Data," filling missing values with 0 might be appropriate if missing values indicate zero sales or if zero is a meaningful value in the context of the dataset.

inplace=True Parameter:

- The inplace=True parameter modifies the original DataFrame directly, avoiding the need to create a new DataFrame with the cleaned data.

- It is a convenient option when you want to make changes to the existing DataFrame without creating a separate copy.

Fixing Delimiters in “Customer Data”

```
df_customers = pd.read_csv(io.BytesIO(uploaded['Customers.csv']), delimiter='\t')
```

Figure 7 - Fixing Delimiters in “Customer Data”

This code segment reads the content of the 'Customers.csv' file, which is assumed to be tab-separated, into a Pandas DataFrame named df_customers. The use of io.BytesIO facilitates in-memory reading, making it suitable for scenarios where file uploads are handled without saving the file to disk. The delimiter='\t' parameter ensures that the DataFrame interprets tabs as the column separators in the CSV file.

Reading a CSV File:

- pd.read_csv() is a function from the Pandas library used to read data from a CSV (Comma-Separated Values) file into a Pandas DataFrame, which is a two-dimensional tabular data structure.
- The DataFrame is essentially a table with rows and columns, allowing for efficient data manipulation and analysis.

Using io.BytesIO for In-Memory Reading:

- io.BytesIO(uploaded['Customers.csv']) is an in-memory file-like object that allows reading the content of the CSV file without the need for a physical file on disk.
- This is particularly useful in environments like Google Colab, where file uploads are handled in-memory.

Specifying the Delimiter:

- delimiter='\t' is an optional parameter that specifies the delimiter used in the CSV file. In this case, it is set to a tab ('\t') character.
- The delimiter determines how the columns in the CSV file are separated. Common delimiters include commas (,), tabs ('\t'), or semicolons (';').

Assigning the Result to a DataFrame:

- df_customers = pd.read_csv(io.BytesIO(uploaded['Customers.csv']), delimiter='\t') combines the steps above by reading the CSV content using the specified delimiter and assigning the result to a DataFrame named df_customers.

Purpose of delimiter='\t':

- The choice of delimiter ('\t') is based on the assumption that the data in the CSV file is tab-separated. This is often the case when data is exported or saved in a format where tabs are used to separate values instead of commas.

Cleaning "Sales Data" Columns

```
[29] sales_years = ['1994', '1995', '1996', '1997', '1998', '1999', '2000', '2011', '2012', '2013', '2014', '2015', '2016', '2017', '2018', '2019', '2020']

     for year in sales_years:
         df_sales[year] = pd.to_numeric(df_sales[year], errors='coerce')
```

Figure 8 - Cleaning "Sales Data" Columns

This code segment iterates through each year in the list sales_years, converts the corresponding columns in the "Sales Data" DataFrame to numeric format, and handles any conversion errors by replacing non-numeric values with NaN. This is a common preprocessing step to ensure data consistency and prepare the DataFrame for numerical analysis.

List of Years:

- sales_years = ['1994', '1995', ..., '2020'] defines a list named sales_years containing strings representing the years relevant to the "Sales Data" DataFrame.
- Each string corresponds to a column name in the DataFrame.

Loop through Years:

- for year in sales_years: initiates a loop that iterates over each element (year) in the sales_years list.
- The loop allows for performing the same set of operations on each year.

Convert Column to Numeric:

- df_sales[year] = pd.to_numeric(df_sales[year], errors='coerce') uses the pd.to_numeric() function to convert the values in the column corresponding to the current year to numeric format.
- The errors='coerce' parameter handles cases where conversion is not possible by replacing non-numeric values with NaN (Not a Number).

Purpose of Conversion:

- The primary purpose of this conversion is to ensure that the data in each yearly column is treated as numeric. This is important for subsequent mathematical and analytical operations.
- Some columns may contain non-numeric values or missing data, and the conversion with pd.to_numeric() helps address these issues by converting valid numeric values and handling errors gracefully.

Data Exploration

visual representation of the distribution of different education levels Explore "Customer Data"

```
[30] # Example: Distribution of 'Education'
     sns.countplot(x='Education', data=df_customers)
     plt.title('Distribution of Education Levels')
     plt.show()
```

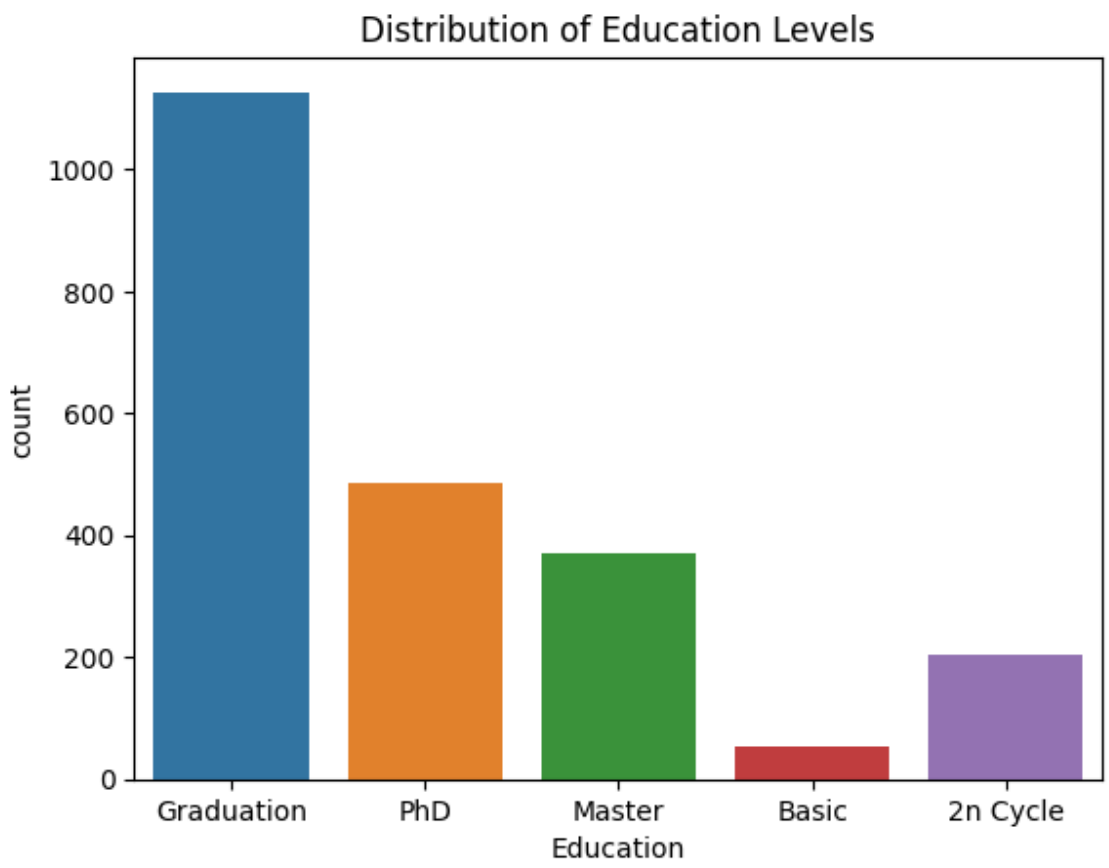


Figure 9 - visual representation of the distribution of different education levels Explore "Customer Data"

This code segment employs Seaborn to create a count plot showcasing the distribution of education levels in the "Customer Data" DataFrame. The resulting visual representation helps in understanding the prevalence of different education levels among the customers in the dataset.

Creating a Count Plot:

- The sns.countplot(x='Education', data=df_customers) line utilizes the Seaborn library (sns) to generate a count plot. This type of plot is particularly useful for displaying the frequency or count of different categories in a categorical variable.
- In this case, the 'Education' column from the "Customer Data" DataFrame (df_customers) is used for the horizontal (x-axis) distribution of categories.

Seaborn Count Plot Functionality:

- Seaborn simplifies the process of creating statistical visualizations, and the countplot function automatically counts the occurrences of each category and plots them as bars.

Specifying the Plot Title:

- plt.title('Distribution of Education Levels') sets a title for the count plot. The title provides a brief description of what the plot illustrates, in this case, the distribution of education levels.

Displaying the Plot:

- The plt.show() function is used to display the generated plot. Once executed, the count plot will be visible to the user.

Purpose of the Plot:

- The primary objective of this count plot is to visually represent how the different education levels are distributed among the customers in the dataset.
- Each bar on the plot corresponds to a specific education level, and the height of the bar indicates the frequency or count of customers with that particular education level.

Interpreting the Plot:

- Observing the count plot allows for a quick assessment of the proportions of customers falling into various education categories. It provides insights into the educational diversity within the customer base.

creating and displaying a line plot Explore "Sales Data"

```
fig, ax = plt.subplots(figsize=(12, 6))
sns.lineplot(x='Year', y='Sales', data=melted_sales, ax=ax)
plt.title('Total Sales Over the Years')
plt.show()
```

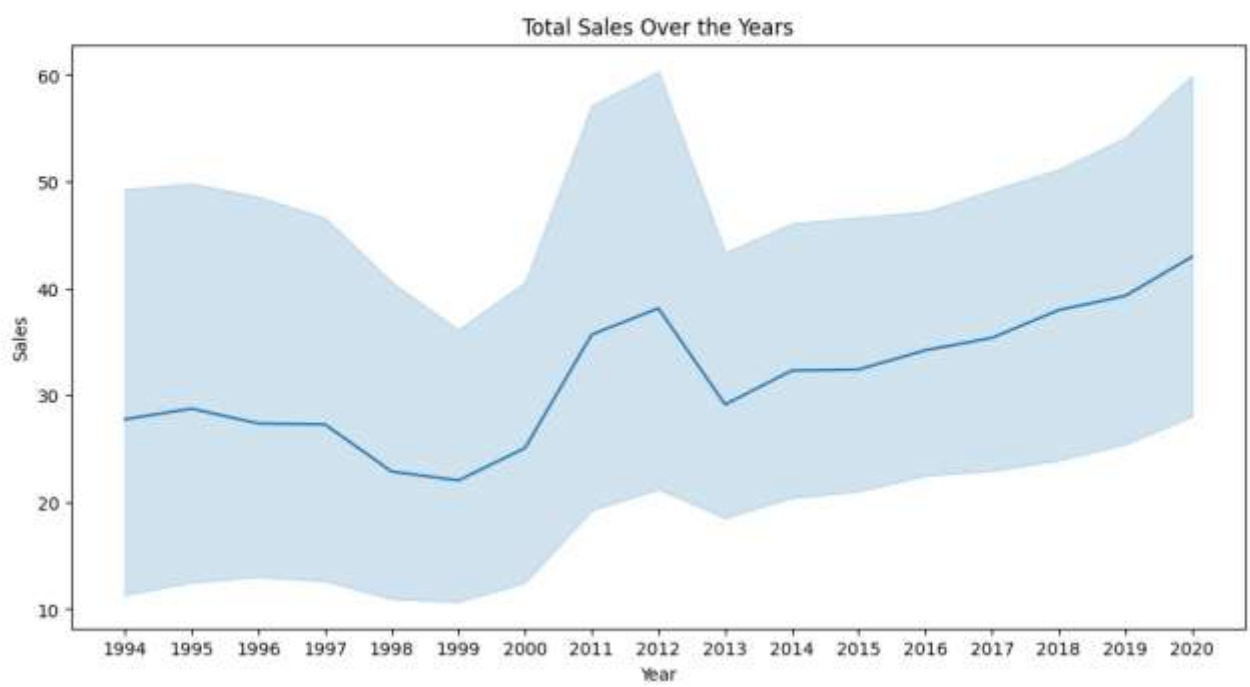


Figure 10 - creating and displaying a line plot Explore "Sales Data"

This code segment creates a line plot using Seaborn to visualize the trend in total sales over a series of years. The plot is displayed with a specified figure size and title, providing a clear and informative representation of how sales have evolved over time.

Creating a Subplots Figure:

- The line fig, ax = plt.subplots(figsize=(12, 6)) uses Matplotlib to create a figure (fig) and an axis (ax). The figsize parameter specifies the dimensions of the figure, setting it to 12 units in width and 6 units in height.

Seaborn Line Plot:

- The line sns.lineplot(x='Year', y='Sales', data=melted_sales, ax=ax) utilizes Seaborn to generate a line plot. The 'Year' column is plotted on the x-axis, and 'Sales' is plotted on the y-axis.
- The data=melted_sales parameter specifies that the data for the plot is taken from the "melted_sales" DataFrame. The ax=ax parameter ensures that the plot is drawn on the previously created axis (ax).

Setting Plot Title:

- The line plt.title('Total Sales Over the Years') sets a title for the plot. The title provides a brief description of what the plot illustrates, which, in this case, is the trend in total sales over the specified years.

Displaying the Plot:

- The line plt.show() is used to display the generated line plot on the previously created figure and axis.

Purpose of the Plot:

- The main purpose of this code is to visually represent and communicate how total sales have changed over a sequence of years.

- A line plot is chosen as it allows for a clear observation of trends or patterns in sales data, making it easier to understand the overall trajectory.

Adjusting Figure Size:

- The use of figsize=(12, 6) in plt.subplots allows for customization of the dimensions of the plot, ensuring it is appropriately sized for clarity.

Creating and displaying a correlation heatmap

```
# Example: Correlation Heatmap for "Customer Data"
fig, ax = plt.subplots(figsize=(26, 20))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", ax=ax)
plt.title('Correlation Heatmap - Customer Data')
plt.show()
```

Figure 11 - Creating and displaying a correlation heatmap

This code segment creates and displays a correlation heatmap using Seaborn for the "Customer Data" DataFrame. The resulting plot provides valuable insights into the relationships between numerical variables, aiding in exploratory data analysis.

Creating a Subplots Figure:

- The line fig, ax = plt.subplots(figsize=(26, 20)) utilizes Matplotlib to create a figure (fig) and an axis (ax). The figsize parameter specifies the dimensions of the figure, setting it to 26 units in width and 20 units in height.

Seaborn Heatmap:

- The line sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", ax=ax) uses Seaborn to generate a heatmap for the correlation matrix (corr_matrix) of the "Customer Data" DataFrame.
- The annot=True parameter adds numerical annotations to the heatmap, displaying the correlation coefficients on each cell.
- The cmap='coolwarm' parameter sets the color map for the heatmap, using cool and warm colors to represent negative and positive correlations, respectively.
- The fmt=".2f" parameter formats the displayed values with two decimal places.
- The ax=ax parameter ensures that the heatmap is drawn on the previously created axis (ax).

Setting Plot Title:

- The line plt.title('Correlation Heatmap - Customer Data') sets a title for the heatmap. The title provides a brief description of what the plot illustrates, which, in this case, is the correlation structure of the variables in the "Customer Data" DataFrame.

Displaying the Plot:

- The line plt.show() is used to display the generated heatmap on the previously created figure and axis.

Purpose of the Plot:

- The main purpose of this code is to visually represent the correlation relationships between different numerical variables in the "Customer Data" dataset.
- A heatmap is an effective way to quickly identify patterns of positive or negative correlations, helping to understand how variables are related to each other.

Adjusting Figure Size:

- The use of figsize=(26, 20) in plt.subplots allows for customization of the dimensions of the heatmap, ensuring it is appropriately sized for detailed examination.

Evaluating the importance of data analytical techniques to the decision-making process

Data analytics plays a crucial role in enhancing business decision-making processes. One significant aspect is the identification of customer needs and preferences. By analyzing customer data, such as purchase history and website interactions, businesses gain valuable insights into customer behaviors. For example, a cosmetic brand utilized data visualization to identify consumer personas during the COVID-19 pandemic, resulting in a 156% increase in brand engagement.

Another key benefit is optimizing operations. Data analytics helps in strategic decision-making, such as determining ideal locations for physical stores based on factors like footfall and market density. A local commercial bank, facing competition, utilized location analytics to identify non-performing branches and integrate insights into their branch transformation blueprint.

Improving marketing strategies is another area where data analytics excels. By analyzing campaign performance, businesses can tailor their strategies for better results. An American footwear company achieved remarkable success by combining consumer insights and creative strategies, leading to a 47 times increase in sales in Singapore.

Data analytics also aids in predicting trends and market changes. Retailers, for instance, can use data to track seasonal buying patterns and anticipate shifts in consumer behavior. A quick-

service restaurant chain in Thailand used data sets and geolocation data to predict consumer behavior, resulting in a 12% increase in daily sales.

The ability to make data-driven decisions is a fundamental advantage of data analytics. Financial service companies, for example, can monitor customer spending patterns to identify potential fraud. During the COVID-19 pandemic, a transportation service leader in Indonesia utilized data analysis to understand commuter patterns and optimize route planning.

To become a data-driven business, a clear roadmap is essential. This involves defining objectives, fostering a data culture, establishing robust data collection mechanisms, creating data warehousing, developing data analytics capabilities, identifying key performance indicators (KPIs), implementing data-driven decision-making processes, using data visualization, continuous improvement, leadership support, prioritizing data privacy and security, fostering collaboration, and staying updated on industry trends.

Business analytics is not just about making better decisions; it's about staying ahead of the competition. Analyzing consumer patterns through segmentation, forecasting, pricing, and customer satisfaction enables businesses to provide the right products and services, resulting in increased profits and customer satisfaction.

Moreover, data-driven decisions contribute to internal performance improvements. Small businesses can streamline operations, reduce operational costs, and enhance product development by leveraging data insights. Analytics also plays a crucial role in risk mitigation, providing businesses with valuable information to predict and plan for potential risks.

In conclusion, the importance of data analytics in decision-making cannot be overstated. It empowers businesses to understand their customers, optimize operations, improve marketing strategies, predict market changes, and make informed, data-driven decisions. As a disruptive technology, businesses must embrace data analytics to stay competitive in today's dynamic market. Using data analytics is like having a superpower for businesses. It helps them understand what customers want, improve how they work, and do better in marketing. For example, a makeup brand got 156% more people interested during COVID-19 by looking at customer data. A bank figured out where to open new branches by using special location

information. Another company selling shoes made 47 times more sales in Singapore by using data smartly. It's not just about making choices; it's about being better than others. To do this, businesses need a clear plan, like setting goals and protecting people's privacy. Data analytics is like a secret weapon that helps businesses stay strong and make smart moves in today's fast-changing world.

Applying an appropriate tool or programming language to demonstrate these descriptive analytics techniques

Descriptive analysis involves summarizing and describing the main features of a dataset. In a Python environment like Google Colab, the Pandas library is commonly used for data manipulation and analysis. Below is a general outline of how you can perform descriptive analysis for the given dataset. Please note that specific analyses may vary based on the structure and characteristics of the dataset.

Importing and Reading the CSV files

```
[1] import pandas as pd
    import numpy as np
    import matplotlib.pyplot as plt
    import seaborn as sns
```

```
[2] from google.colab import files
    uploaded = files.upload()
```

Choose Files 2 files

- **Customers.csv**(text/csv) - 220188 bytes, last modified: 11/19/2023 - 100% done
- **Sales.csv**(text/csv) - 19294 bytes, last modified: 11/28/2023 - 100% done

```
Saving Customers.csv to Customers.csv
Saving Sales.csv to Sales.csv
```

Figure 12 - Importing and Reading the CSV files

The provided code segment is written in Python and utilizes various libraries for data analysis and visualization. Firstly, it imports essential libraries such as pandas (as pd) for data manipulation, numpy (as np) for numerical operations, and matplotlib.pyplot and seaborn for data visualization. The code also includes a command specific to Google Colab, a cloud-based platform for coding, which allows users to upload files interactively. In this case, it uses the "files" module from Google Colab to upload files. The term "uploaded" is assigned the result of the "files.upload()" function, which prompts the user to upload files. The uploaded files could be datasets or any other files needed for analysis. This code segment sets the stage for further

data analysis and visualization by preparing the environment with the necessary tools and potentially loading datasets for subsequent processing.

Display basic information about the datasets

```
# Display basic information about the datasets
print("Customers Data Overview:")
print(df_customers.info())
```

```
Customers Data Overview:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 1 columns):
 #   Column
---  ------
 0   ID Year_Birth      Education       Marital_Status  Income  Kidhome Teenhome        Dt_Customer
dtypes: object(1)
memory usage: 17.6+ KB
None
```

Figure 13 - Display basic information - Customer data

The provided code segment uses the "print" function in Python to display basic information about a dataset referred to as "Customers Data." The term "df_customers" likely represents a pandas DataFrame containing customer-related information. The "info()" method is applied to this DataFrame, which provides an overview of its structure and content. The output includes details such as the total number of entries (rows), the data types of each column, and the count of non-null values in each column. This information is valuable for understanding the dataset's dimensions, identifying any missing values, and gaining insights into the types of data present. Displaying this basic dataset overview is a common initial step in the exploratory data analysis process to inform subsequent analytical decisions.

```
[5] print("\nSales Data Overview:")
    print(df_sales.info())
```

```
Sales Data Overview:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 157 entries, 0 to 156
Data columns (total 30 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Category 1  157 non-null    object
 1   Quality     157 non-null    object
 2   Country     157 non-null    object
 3   1994        62 non-null     object
 4   1995        62 non-null     object
 5   1996        66 non-null     object
 6   1997        70 non-null     object
 7   1998        90 non-null     object
 8   1999        98 non-null     object
 9   2000        95 non-null     object
 10  2001        96 non-null     object
 11  2002        95 non-null     object
 12  2003        96 non-null     object
 13  2004        104 non-null    object
 14  2005        115 non-null    object
 15  2006        120 non-null    object
 16  2007        118 non-null    object
 17  2008        117 non-null    object
 18  2009        116 non-null    object
 19  2010        116 non-null    object
 20  2011        117 non-null    object
 21  2012        120 non-null    object
 22  2013        119 non-null    object
```

Figure 14 - Display basic information - Sales data

This code segment employs the "print" function in Python to present a concise overview of a dataset referred to as "Sales Data." Presumably, the variable "df_sales" denotes a pandas DataFrame containing information related to sales. The "info()" method is applied to this DataFrame, generating a summary of essential details about the dataset. The output includes information on the total number of entries (rows), the data types associated with each column, and the count of non-null values in each column. By displaying this overview, users can quickly ascertain key characteristics of the sales dataset, such as its size, the types of data it contains, and whether there are any missing values. This step is crucial in the data analysis process as it

helps in understanding the structure and quality of the dataset, guiding subsequent analytical actions.

Filling Missing Values

```
# Remove rows with missing values
df_customers.dropna(inplace=True)
df_sales.fillna('OPM EMPTY', inplace=True)  # Fill missing values with 0 in the sales data
df_sales
```

	Category 1	Quality	Country	1994	1995	1996	1997	1998	1999	2000	...	2011
0	OPM EMPTY	All	Total	OPM EMPTY	1,581.9	1,606.7	1,701.0	1,839.4	1,933.5	2,124.1	...	3,860.4
1	OPM EMPTY	Standard	Total	OPM EMPTY	1,438.8	1,461.8	1,546.9	1,667.1	1,752.7	1,917.1	...	3,618.3
2	OPM EMPTY	Standard	United States	243.0	214.0	219.5	225.0	239.5	249.0	259.0	...	803.0
3	OPM EMPTY	Standard	Ireland	537.5	546.0	513.0	524.5	526.0	517.5	536.4	...	473.4
4	OPM EMPTY	Standard	France	92.0	118.0	137.0	151.0	161.0	181.0	225.0	...	370.3

Figure 15- Filling Missing Values All together

In this code segment, data cleaning operations are applied to two DataFrames, likely named "df_customers" and "df_sales." Firstly, the method `dropna()` is used on the "df_customers" DataFrame with `inplace =True` to remove rows that contain any missing values. This action effectively eliminates entries with incomplete or undefined data, enhancing the overall data quality.

Subsequently, the "fillna()" method is employed on the "df_sales" DataFrame to address missing values in the dataset. In this case, the missing values are replaced with the string "OPM EMPTY." This method is useful for handling missing data by substituting them with a designated

placeholder, making it easier to identify and manage such instances during subsequent analyses. The use of `inplace=True` ensures that these modifications are applied directly to the original DataFrames. These cleaning steps contribute to the overall data preparation process, ensuring that the datasets are more suitable for downstream analysis and interpretation.

```
df3= df_sales2
df3["1994"]= df3 ['1994'].fillna ("OPM EMPTY")
df3
```

	Category 1	Quality	Country	1994	1995	1996	1997	1998	1999	2000	...	2011
0	Wine	All	Total	1,535.2	1,581.9	1,606.7	1,701.0	1,839.4	1,933.5	2,124.1	...	3,860.4
1	Wine	Standard	Total	1,404.8	1,438.8	1,461.8	1,546.9	1,667.1	1,752.7	1,917.1	...	3,618.3
2	Wine	Standard	United States	243.0	214.0	219.5	225.0	239.5	249.0	259.0	...	803.0
3	Wine	Standard	Ireland	537.5	546.0	513.0	524.5	526.0	517.5	536.4	...	473.4
4	Wine	Standard	France	92.0	118.0	137.0	151.0	161.0	181.0	225.0	...	370.3
...	...	...	...	...	...	...	...	...	...	...	...	...
152	Wine	Super Premium	DF Singapore	OPM EMPTY	NaN	NaN	NaN	0.1	0.1	0.1	...	NaN
153	Wine	Super Premium	DF South Korea	OPM EMPTY	NaN	NaN	NaN	NaN	0.3	0.2	...	NaN
154	Wine	Super Premium	DF Switzerland	OPM EMPTY	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN
155	Wine	Super Premium	DF Turkey	OPM EMPTY	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN
156	Wine	Super Premium	DF US Pacific	OPM EMPTY	NaN	NaN	NaN	0.1	0.2	0.1	...	NaN

157 rows × 30 columns

Figure 16 - Filling Missing Values Individually

In this code segment, a new DataFrame named "df3" is created, presumably as a copy of an existing DataFrame named "df_sales2." The operation `df3["1994"] = df3['1994'].fillna("OPM EMPTY")` is then performed, focusing on the column labeled "1994" within the "df3" DataFrame. The "fillna" method is applied to this specific column, which is a pandas function used for filling missing values. In this case, the missing values (NaN, if any) in the "1994" column are replaced with the string "OPM EMPTY." The resulting DataFrame, "df3," now contains the modified "1994" column, where any previously missing values have been substituted with the

specified string. This approach is a common technique in data preprocessing to handle missing data by replacing it with a recognizable placeholder or value, facilitating further analysis or visualization.

Checking Duplicated Values

```
[11] # Checking for duplicate rows
     df_customers.drop_duplicates(inplace=True)
```

```
[45] df3.duplicated().sum()

     0
```

```
df3.duplicated()

0      False
1      False
2      False
3      False
4      False
       ...
152    False
153    False
154    False
155    False
156    False
Length: 157, dtype: bool
```

Figure 17 - Checking and removing Duplicated values

In this code segment, duplicate rows in two DataFrames, likely named "df_customers" and "df3," are addressed to ensure data integrity. First, for the "df_customers" DataFrame, the method `drop_duplicates()` is employed with `inplace=True` to remove any identical rows, retaining only the unique entries. This action helps eliminate redundancy in the customer data, ensuring that each row is distinct based on its values.

Subsequently, the code checks for duplicate rows in the "df3" DataFrame using the `duplicated()` method. The expression `df3.duplicated().sum()' calculates the total count of duplicated rows in "df3," providing a numerical summary of the number of identical entries. The boolean result of `df3.duplicated()` indicates, for each row, whether it is a duplicate of a previous row (True) or not (False). This verification process is essential in data quality assurance, as it helps maintain the uniqueness and reliability of the datasets, preventing potential inaccuracies that could arise from redundant information.

Displaying the data

```
df3
```

	Category 1	Quality	Country	1994	1995	1996	1997	1998	1999	2000	...	2011
0	Wine	All	Total	1,535.2	1,581.9	1,606.7	1,701.0	1,839.4	1,933.5	2,124.1	...	3,860.4
1	Wine	Standard	Total	1,404.8	1,438.8	1,461.8	1,546.9	1,667.1	1,752.7	1,917.1	...	3,618.3
2	Wine	Standard	United States	243.0	214.0	219.5	225.0	239.5	249.0	259.0	...	803.0
3	Wine	Standard	Ireland	537.5	546.0	513.0	524.5	526.0	517.5	536.4	...	473.4
4	Wine	Standard	France	92.0	118.0	137.0	151.0	161.0	181.0	225.0	...	370.3
...	...	...	...	...	...	...	...	...	...	...	...	...

```
df_customers2
```

	ID\tYear_Birth\tEducation\tMarital_Status\tIncome\tKidhome\tTeenhome\tDt_Customer\tRecency\tMntWi
0	
1	

Figure 18 - Displaying Data

The above python code will call the appointed function and show the data.

Demonstrating the above descriptive analytics techniques.

Descriptive analytics involves summarizing and presenting data to gain insights into its key characteristics. In the provided code segments, we witness the foundational steps of descriptive analytics, beginning with the import and examination of datasets related to customers and sales. This process sets the stage for subsequent analyses to understand the structure, cleanliness, and basic statistics of the data.

The above codes display basic information about the customer and sales datasets, such as the number of entries, data types, and counts of non-null values. This step is crucial for gaining an initial understanding of the datasets' dimensions and potential data quality issues. By using the "info ()" method in Python, the code provides concise overviews that aid in identifying missing values and formulating strategies for data cleaning.

Data cleaning is a fundamental aspect of descriptive analytics. The code demonstrates techniques for handling missing values in both the customer and sales datasets. The "dropna()" method is used to remove rows with missing values in the customer dataset, ensuring data completeness. In the sales dataset, the "fillna()" method replaces missing values with a designated placeholder, making it easier to identify and manage such instances during subsequent analyses.

Duplicate values can introduce inaccuracies in analyses, and the code addresses this by checking and handling duplicates. The "drop_duplicates()" method is applied to the customer dataset, removing identical rows and maintaining uniqueness. Additionally, the code checks for duplicate rows in another dataset, providing insights into the prevalence of duplicate entries and supporting data quality assurance efforts.

While the above provided codes focuses on data import, cleaning, and basic information display, the next steps in descriptive analytics often involve data visualization. Visualizations such as histograms, scatter plots, and correlation matrices can offer deeper insights into the distribution and relationships within the datasets. These visualizations contribute to a comprehensive descriptive analytics approach, enabling a more thorough exploration and understanding of the underlying patterns in the data.

probability distribution methods and sampling methods

Probability distribution methods and sampling methods are fundamental concepts in statistics and data analysis, each serving distinct purposes in understanding and modeling data.

Probability Distribution Methods

Probability distribution methods are applied to model the likelihood of different outcomes in a random experiment. The choice of a specific probability distribution depends on the nature of the data and the characteristics of the random variable being analyzed. Some common probability distributions include the normal distribution, binomial distribution, Poisson distribution, and exponential distribution. These methods allow statisticians to describe and quantify the uncertainty associated with random events or variables. For example, the normal distribution is frequently used to model the distribution of continuous data, assuming a symmetric bell-shaped curve.

Sampling Methods

Sampling methods are employed to select a subset of individuals or elements from a larger population, with the goal of making inferences about the population based on the characteristics of the sample. Proper sampling is crucial for generalizing findings from a sample to the entire population. Various sampling methods include simple random sampling, stratified sampling, cluster sampling, and systematic sampling. Simple random sampling involves selecting individuals randomly from the population, ensuring that each member has an equal chance of being chosen. Stratified sampling divides the population into strata or subgroups and then samples from each stratum proportionally. These methods are essential for collecting representative samples and drawing reliable conclusions about population parameters.

In practice, probability distribution methods and sampling methods often work together. After collecting a sample, analysts may use probability distribution methods to model and analyze the characteristics of the sample data. This can provide insights into the uncertainty associated with estimates and help make statistical inferences about the population. Understanding the principles behind these methods is crucial for conducting robust statistical analyses and drawing meaningful conclusions from data.

Probability distribution methods and sampling methods for this dataset

Probability distribution methods are valuable tools in data analysis, helping us understand the likelihood of different outcomes. In our dataset, these methods can be applied to model the probability of various events or values. For instance, if we are examining customer purchase amounts, probability distributions can provide insights into the probability of different spending levels.

One common probability distribution is the normal distribution, often used to model continuous data. In our dataset, we might apply the normal distribution to analyse customer income, assuming a bell-shaped curve. This allows us to estimate the likelihood of income levels and identify the typical income range. For scenarios involving binary outcomes, like whether a customer accepts a promotional offer or not, the binomial distribution could be relevant. It helps us understand the probability of success or failure in a series of independent trials, aiding in predicting customer responses to marketing campaigns. If we are interested in analyzing the number of customer complaints or occurrences of a specific event over a period, the Poisson distribution could be beneficial. It models the probability of a certain number of events happening within a fixed interval.

Sampling methods play a crucial role in drawing meaningful conclusions about a population based on a subset of data. In our dataset, we might be interested in understanding broader customer trends without analyzing every single record. Simple random sampling involves randomly selecting individuals from the dataset, providing each customer an equal chance of being chosen. This method could be applied to obtain a representative sample for general insights into customer behavior or preferences.

Stratified sampling involves dividing the dataset into subgroups (strata) based on certain characteristics. For instance, we might stratify customers based on their geographical location. This allows us to ensure representation from each subgroup, providing more detailed insights. If geographical location is a critical factor in our analysis, cluster sampling could be applied. It involves dividing the population into clusters and randomly selecting entire clusters for analysis. This method can be efficient for studying regional trends in customer behaviour.

Systematic sampling involves selecting every kth item from the dataset after an initial random start. This method is efficient and could be used when there is an inherent order or pattern in the data, such as a chronological ordering of customer interactions

Finally, probability distribution methods and sampling methods offer powerful tools for analysing our dataset. Whether we are modelling the likelihood of specific events using probability distributions or drawing meaningful insights about the entire customer population through strategic sampling, these methods provide a structured approach to making informed decisions based on our data. Understanding and applying these methods contribute to robust statistical analyses and enhance our ability to make reliable inferences about customer behaviour and preferences.

Applying the Probability distribution methods and sampling methods for this dataset

Generating a synthetic dataset of customer income

```python
# Generate synthetic customer income data (replace this with your actual data)
np.random.seed(42)
customer_income = np.random.normal(loc=50000, scale=15000, size=1000)

# Create a DataFrame with customer income
df_customers = pd.DataFrame({'Income': customer_income})

# Display the first few rows of the DataFrame
df_customers.head()
```

	Income
0	57450.712295
1	47926.035482
2	59715.328072
3	72845.447846
4	46487.699379

Figure 19 - Generating a synthetic dataset of customer income

In this code snippet, a synthetic dataset of customer income is generated using the NumPy library in Python. The `np.random.normal` function simulates random numbers drawn from a normal distribution with a specified mean (`loc=50000`) and standard deviation (`scale=15000`). This synthetic income data is then organized into a pandas DataFrame named `df_customers` with a single column labeled 'Income.' The DataFrame is a tabular data structure that facilitates efficient handling and manipulation of the data. Finally, the `head()` method is used to display the first few rows of the DataFrame, providing a glimpse of the generated income data. This initial exploration aids in verifying the structure and content of the dataset, ensuring that the generation process aligns with the intended characteristics of customer income in subsequent analyses.

Creating a histogram to visually depict the distribution of customer income

```
# Plot a histogram to visualize the distribution of customer income
plt.figure(figsize=(8, 6))
sns.histplot(df_customers['Income'], bins=30, kde=True)
plt.title('Distribution of Customer Income')
plt.xlabel('Income')
plt.ylabel('Frequency')
plt.show()
```

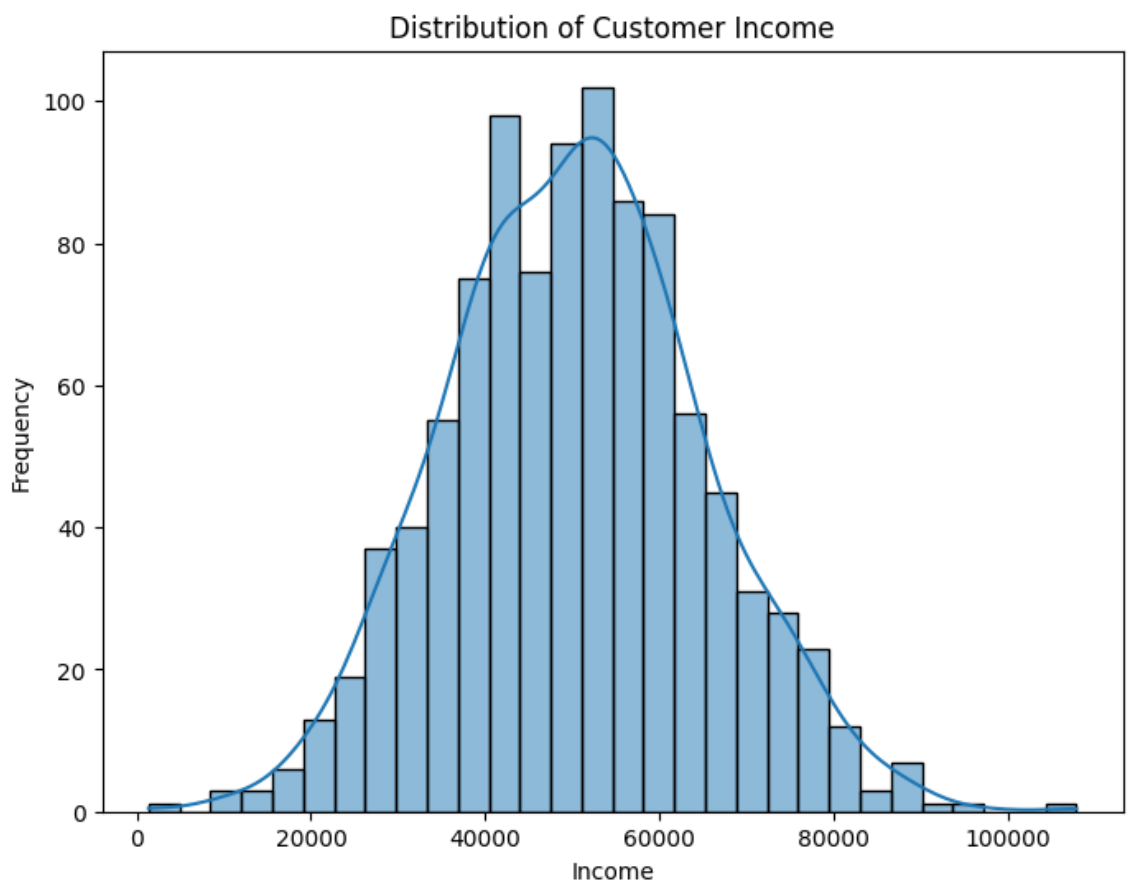


Figure 20 - Creating a histogram to visually depict the distribution of customer income

This code segment creates a histogram to visually depict the distribution of customer income in the synthetic dataset. The `plt.figure(figsize=(8, 6))` sets the size of the resulting plot, and `sns.histplot` from the Seaborn library is employed to generate the histogram. The 'Income'

column from the `df_customers` DataFrame is used as the data source. The `bins=30` parameter specifies the number of bins or intervals for the histogram, enhancing granularity in visualizing income ranges. The `kde=True` option adds a kernel density estimate (KDE) plot to provide a smoothed representation of the probability density function. The title, x-axis label ('Income'), y-axis label ('Frequency'), and overall appearance of the plot are configured using additional Matplotlib functions. This histogram aids in understanding the distribution of customer income values, highlighting patterns or concentrations within specific income ranges in the dataset.

Analysing one qualitative variable

```
# Create a one-way frequency table for the "Category 1" variable
frequency_table = df_sales['Category 1'].value_counts().reset_index()
frequency_table.columns = ['Category 1', 'Frequency']

# Display the frequency table
print(frequency_table)
```

```
  Category 1  Frequency
0       Wine        157
```

Figure 21 - Analysing one qualitative variable

In this code segment, we are generating a one-way frequency table for the "Category 1" variable in the `df_sales` dataset using the Pandas library in Python. The first line of code employs the `value_counts()` method, which tallies the occurrences of each unique value in the "Category 1" column. By doing so, it produces a frequency count for each distinct category, revealing how often each category appears in the dataset. The result is a series of frequencies indexed by the unique categories.

The second part of the code takes this series and transforms it into a structured DataFrame for better presentation and understanding. The `reset_index()` method is utilized to convert the index (containing unique categories) into a regular column. Subsequently, column names are assigned to enhance clarity, where "Category 1" represents the different categories, and "Frequency" indicates the corresponding count for each category. Finally, the frequency table is printed, providing a tabular format that showcases the distribution of counts across various categories within the "Category 1" variable. This tabulated form allows for a more systematic examination of the categorical data and aids in making informed observations about the prevalence of each category in the dataset.

```
[11] # Display counts of unique values for the "Category 1" column
     category_counts = df_sales['Category 1'].value_counts()

     # Plotting the distribution of "Category 1"
     plt.figure(figsize=(10, 6))
     sns.barplot(x=category_counts.index, y=category_counts.values, palette='viridis')
     plt.title('Distribution of Category 1')
     plt.xlabel('Category 1')
     plt.ylabel('Count')
     plt.xticks(rotation=45, ha='right')  # Rotate x-axis labels for better readability
     plt.show()
```

Figure 22 - Analysing one qualitative variable - Bar

In this code segment, we are conducting an analysis of the "Category 1" variable in the `df_sales` dataset using Python with the help of the Pandas and Seaborn libraries. The first line of code utilizes the `value_counts()` method from Pandas to calculate the number of occurrences of each unique value in the "Category 1" column. This essentially generates a tally or frequency count for each distinct category in the dataset, revealing how many times each category appears.

The second part of the code is dedicated to visualizing this frequency information using a bar plot. We use Seaborn's `barplot` function, specifying the x-axis as the unique categories (`category_counts.index`) and the y-axis as the corresponding frequency counts (`category_counts.values`). Additionally, a color palette ('viridis') is applied to the bars for visual appeal. The plot is configured with a title ("Distribution of Category 1") and labeled axes indicating the category and count. To improve readability, the x-axis labels are rotated at a 45-degree angle. The resulting bar plot visually represents the distribution of different categories within the "Category 1" variable, providing a clear overview of their respective frequencies.

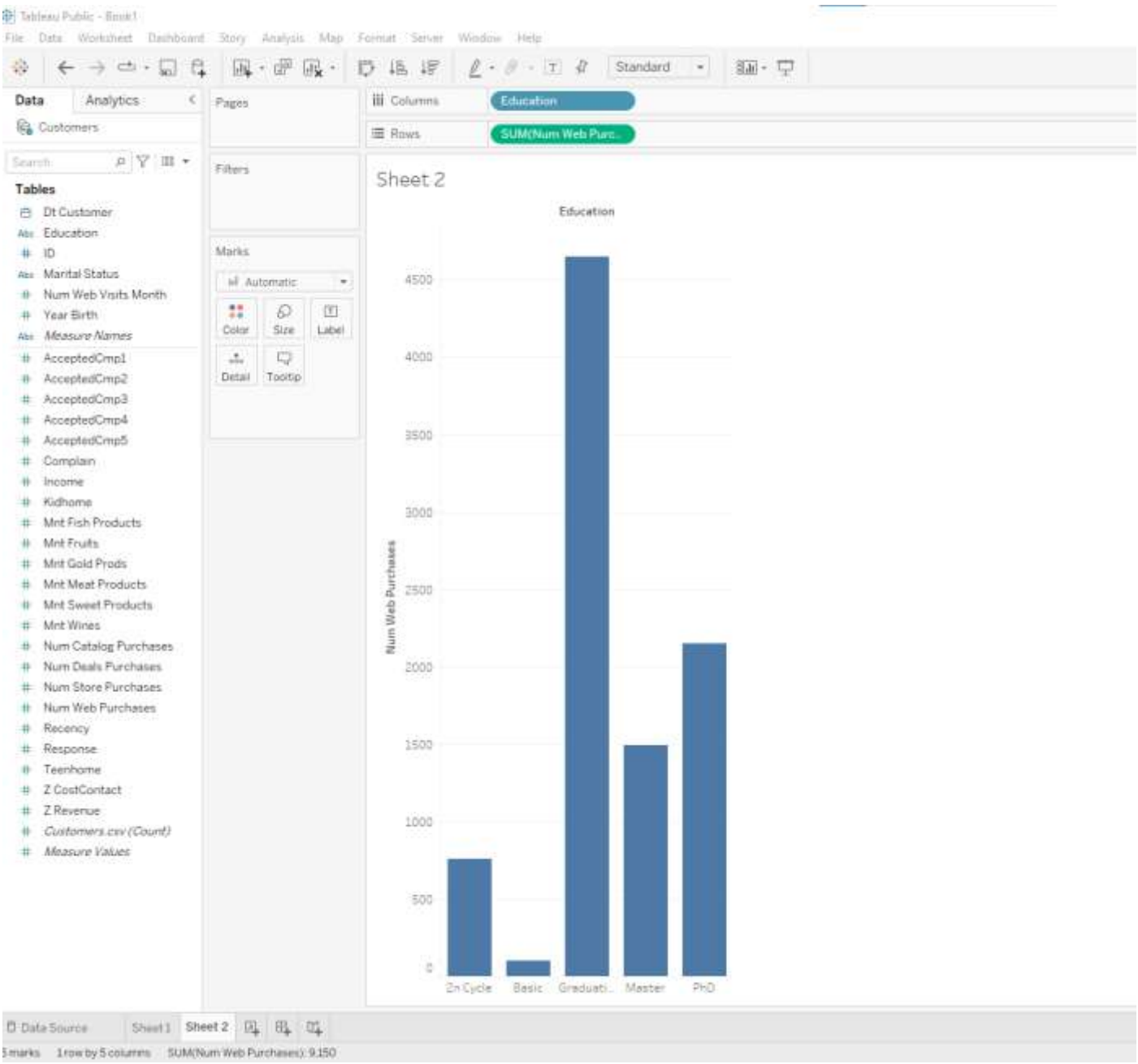


Figure 23 - Analysing one qualitative variable

In Tableau, crafting a one-way frequency table for a categorical variable, like "Education" in the df_customers dataset, involves a straightforward series of steps. Firstly, you launch Tableau Desktop and establish a connection with your dataset, for instance, Customers.csv. By selecting

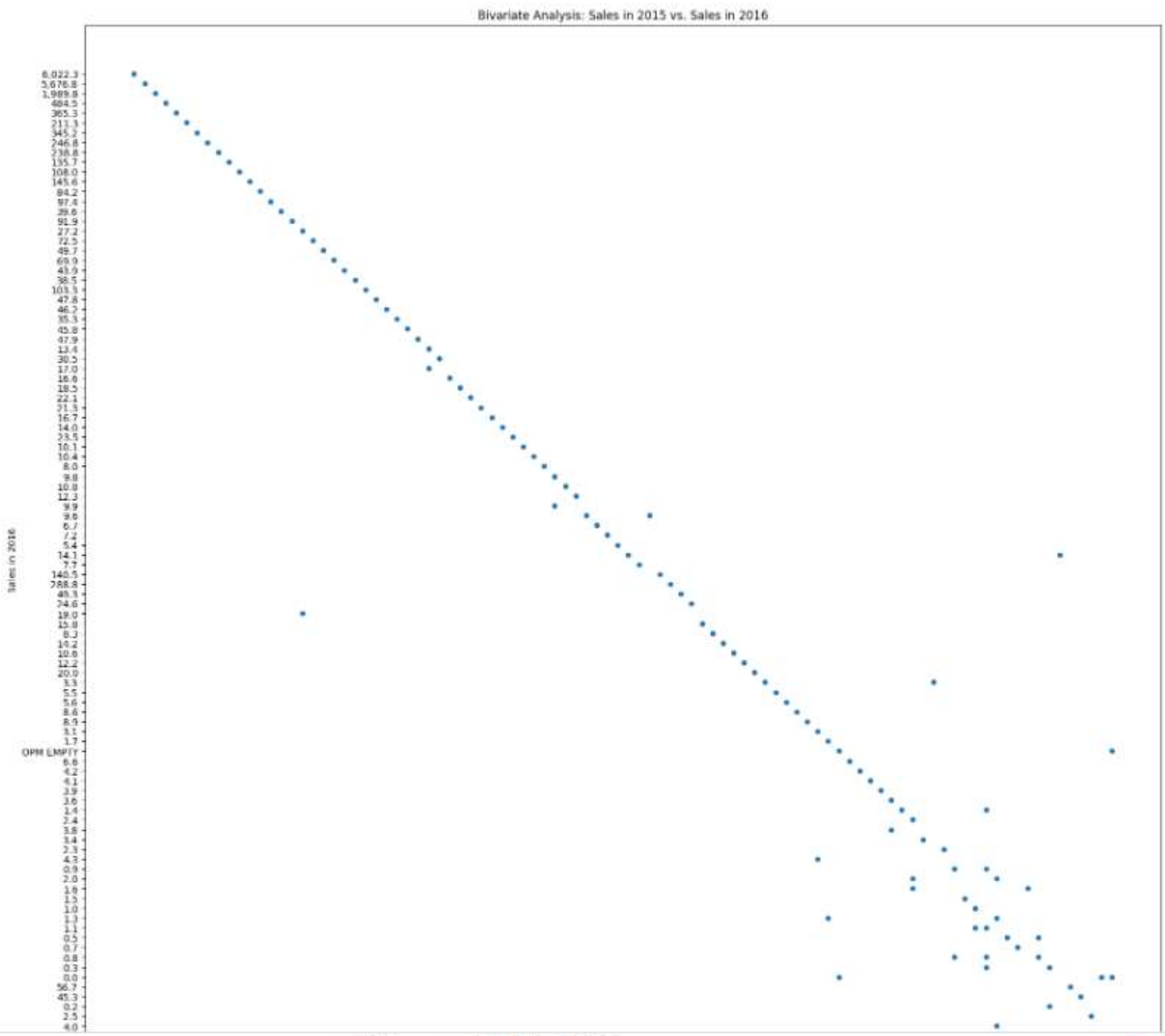
the appropriate file type and indicating the file, you enable Tableau to access and work with your dataset.

Moving on, you construct the frequency table by dragging the "Education" field from the left panel onto the main canvas. Tableau promptly interprets this action and generates a bar chart illustrating the count of each education category. To refine the visual presentation, you have the option to tweak the appearance of the chart, adjusting elements such as color palette, labels, and title for enhanced clarity. Additionally, you can organize the bars by count to create a more orderly and informative display.

Once the visualization is refined, you can interact with the data by clicking on individual bars to explore specific category counts. Leveraging Tableau's interactive features, you can apply filters or highlight particular categories, gaining deeper insights into the dataset. Finally, after completing the analysis, saving your Tableau workbook preserves the visual representation of the one-way frequency table, allowing for easy sharing and collaborative exploration of the qualitative characteristics, such as the distribution of education levels among customers, within the df_customers dataset.

Bivariate analysis of two quantitative variables

```python
# Bivariate analysis: Scatter plot
plt.figure(figsize=(20, 20))
sns.scatterplot(x='2015', y='2016', data=df_sales)
plt.title('Bivariate Analysis: Sales in 2015 vs. Sales in 2016')
plt.xlabel('Sales in 2015')
plt.ylabel('Sales in 2016')
plt.show()
```



3s completed at 7:37 PM

Figure 24 - Bivariate analysysv for sales using python

In the provided code snippet, a bivariate analysis is conducted to explore the relationship between two quantitative variables in the `df_sales` dataset. Specifically, the code creates a scatter plot, a type of graph where each point represents a combination of values for two

variables. The `sns.scatterplot` function from the Seaborn library is utilized for this purpose. The `x` and `y` parameters specify which columns from the DataFrame (`df_sales`) are used for the horizontal and vertical axes of the scatter plot. In this case, the sales data for the year 2015 is represented on the horizontal axis (`x='2015'`), and the sales data for the year 2016 is represented on the vertical axis (`y='2016'`). Each point on the scatter plot corresponds to a pair of sales values for these two years.

The `plt.title`, `plt.xlabel`, and `plt.ylabel` functions are used to provide labels and a title to the plot, enhancing its interpretability. The title indicates the purpose of the analysis, stating that it's a "Bivariate Analysis: Sales in 2015 vs. Sales in 2016." The x-axis label specifies the variable being represented on the horizontal axis ("Sales in 2015"), and the y-axis label specifies the variable being represented on the vertical axis ("Sales in 2016"). By visualizing the scatter plot, one can gain insights into the potential relationship or pattern between sales in these two specific years. If the points tend to form a recognizable shape, it may suggest a correlation or trend between the sales figures for the two years.

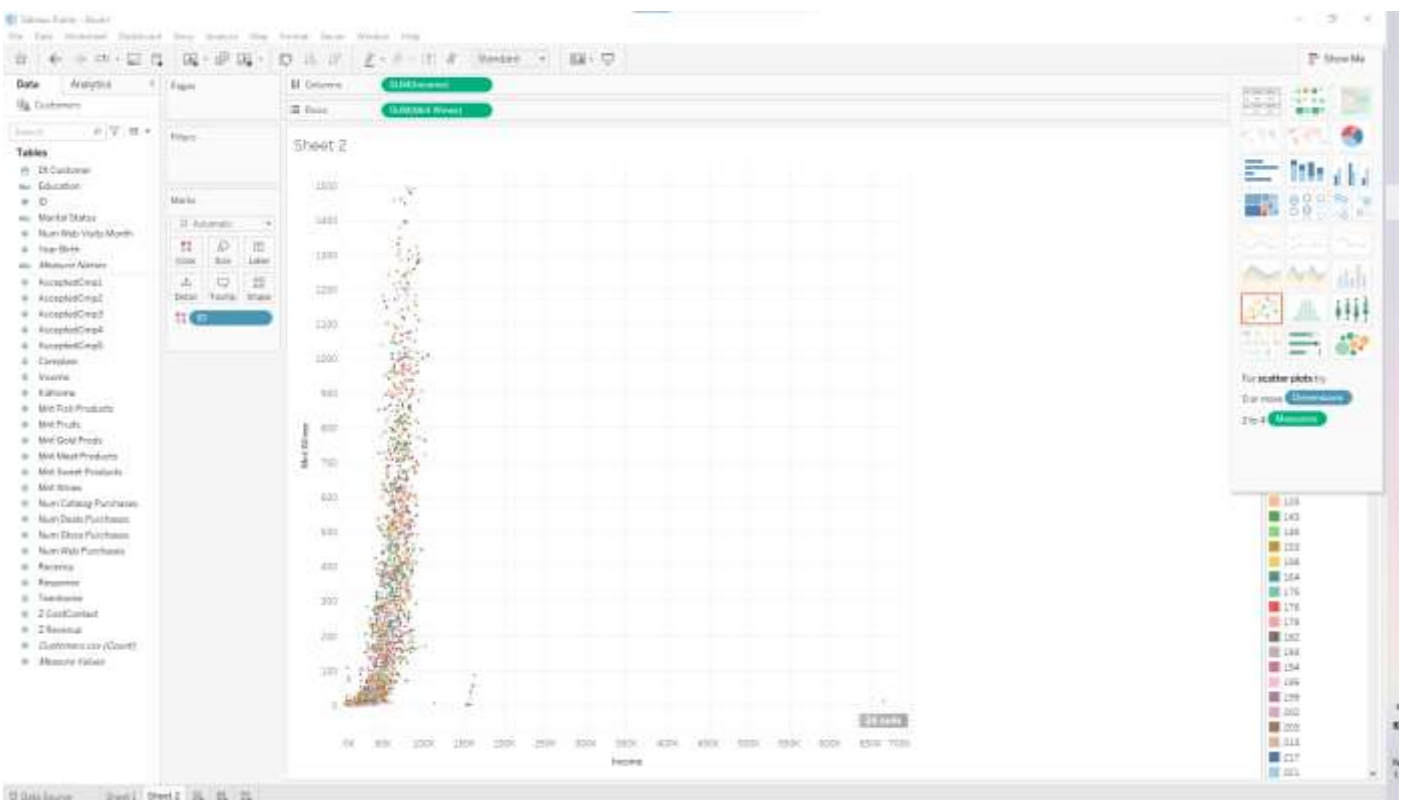


Figure 25 - bivariate variables using tableu

Using Tableu the execution of a bivariate analysis is facilitated by creating a scatter plot to examine the interrelation between two quantitative variables. To initiate this process, the dataset (such as Customers.csv) is imported into Tableau by establishing a connection to the respective file. Subsequently, the construction of the scatter plot is achieved by dragging the "Income" field to the Columns shelf and the "MntWines" field to the Rows shelf. The resulting scatter plot intuitively portrays the variable "Income" along the x-axis and "MntWines" along the y-axis.

To refine the visual representation, Tableau provides customization options for various elements. Additional fields, such as "Education" or "Marital_Status," can be integrated into the analysis by incorporating them into the "Color" shelf, thereby facilitating the differentiation of data points by categorical distinctions. Moreover, introducing another quantitative variable, like "NumDealsPurchases," to the "Size" shelf enables the depiction of data point sizes, enriching the analytical depth of the visualization.

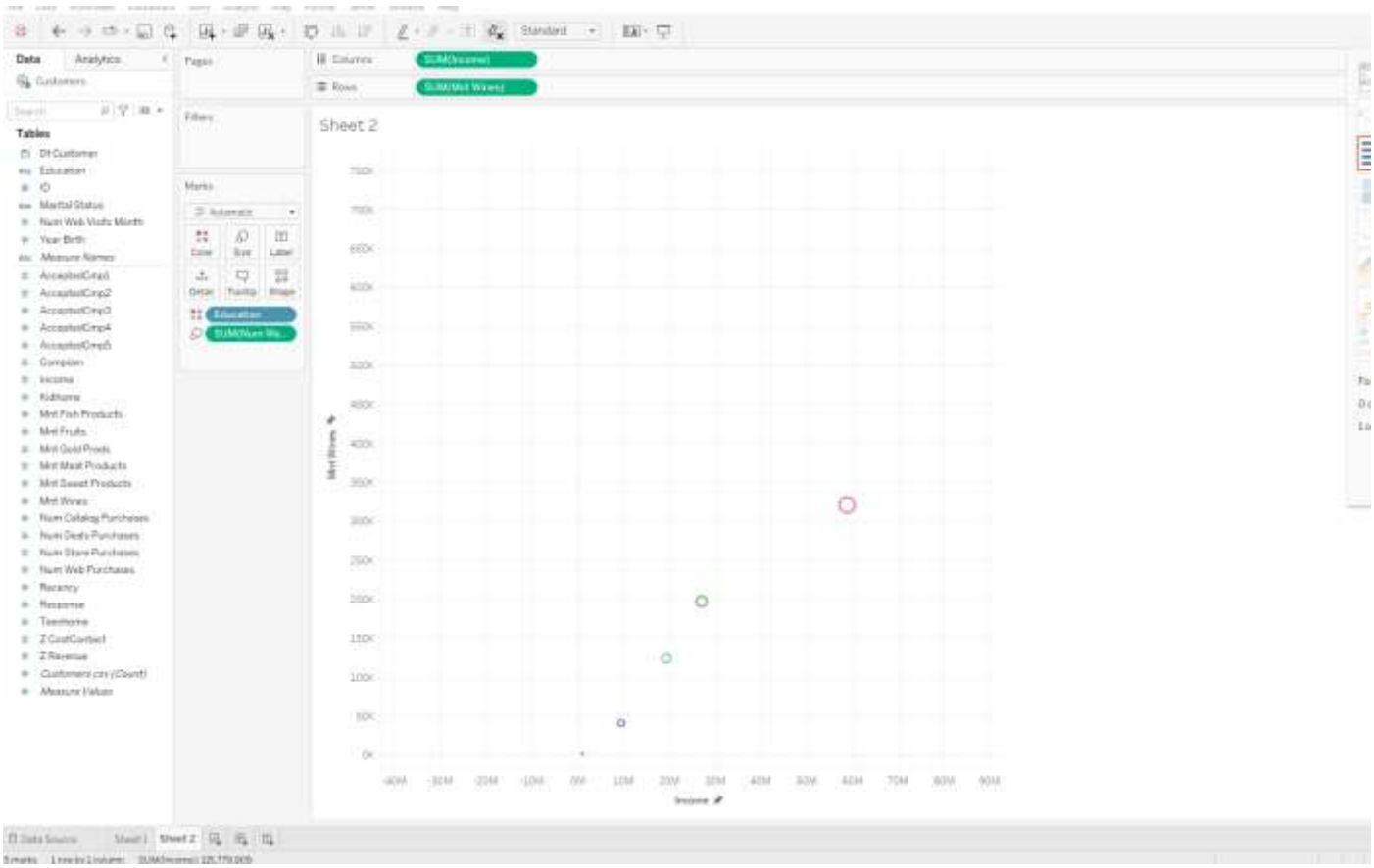


Figure 26 -Bivariate analysis of two quantitative variables

In Tableau, we can explore more aspects of the `df_customers` dataset by considering the connections between "Income," "MntWines," and "NumWebPurchases." To start, load your data into Tableau and build a scatter plot by placing "Income" on the horizontal axis and "MntWines" on the vertical axis. Additionally, use the "NumWebPurchases" variable to adjust the size of data points for a more detailed view. Enhance the visualization by adding categorical details like "Education" or "Marital_Status" to color-coded categories, and customize tooltips to display information when hovering over data points.

Tableau's interactive features enable dynamic exploration. Click on data points to highlight specific information and use filters to focus on particular aspects, such as observations with a specific "Education" level or "Marital_Status." Once satisfied with the analysis, save your Tableau workbook to share these insights seamlessly. By considering multiple variables, this

approach provides a richer understanding of how different factors interact and influence one another, making complex relationships more accessible and actionable for decision-making.

Analysing Data in Customers dataset using Tableu

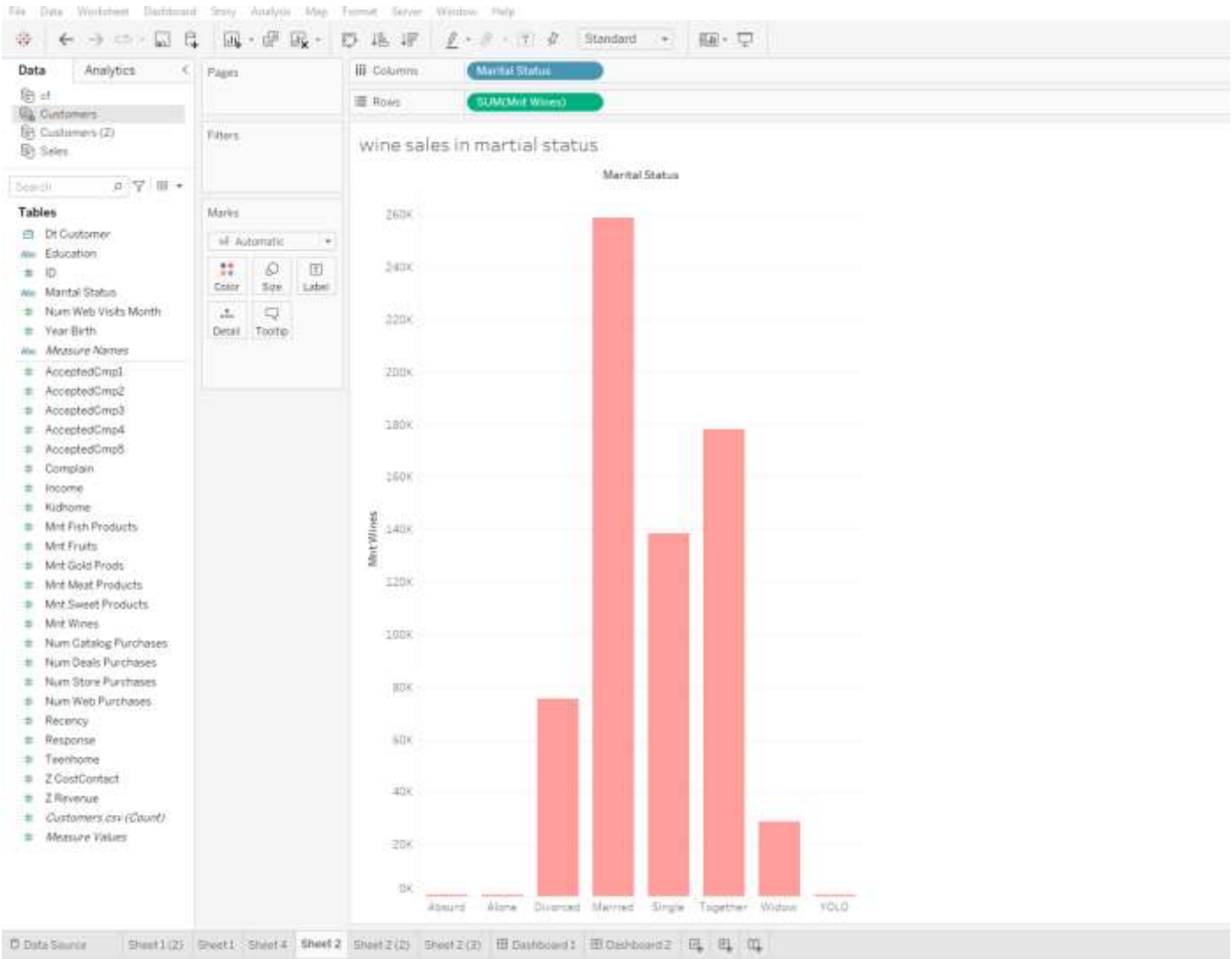


Figure 27 - Wines sales in martial status

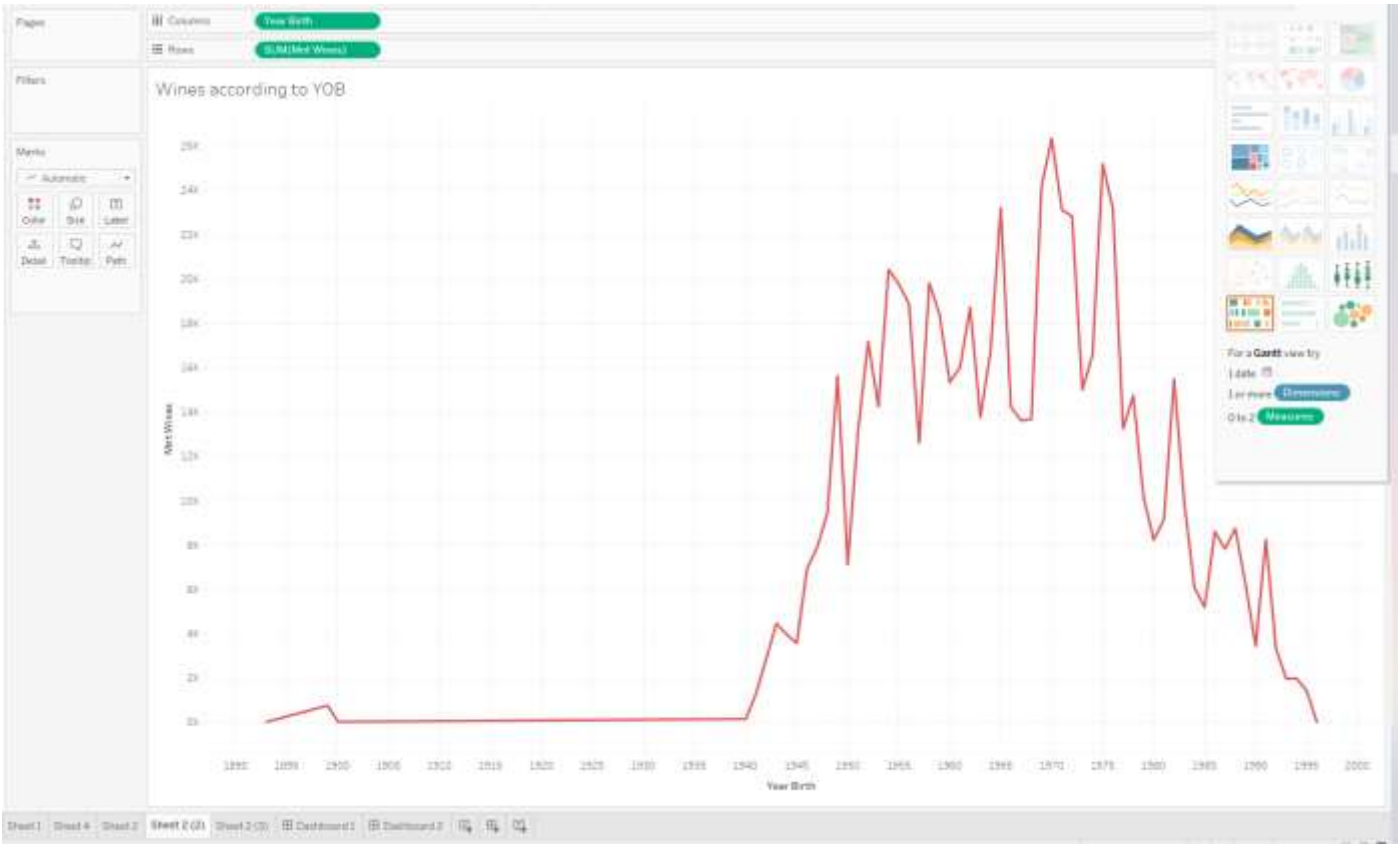


Figure 28 - Wines sold according to YOB

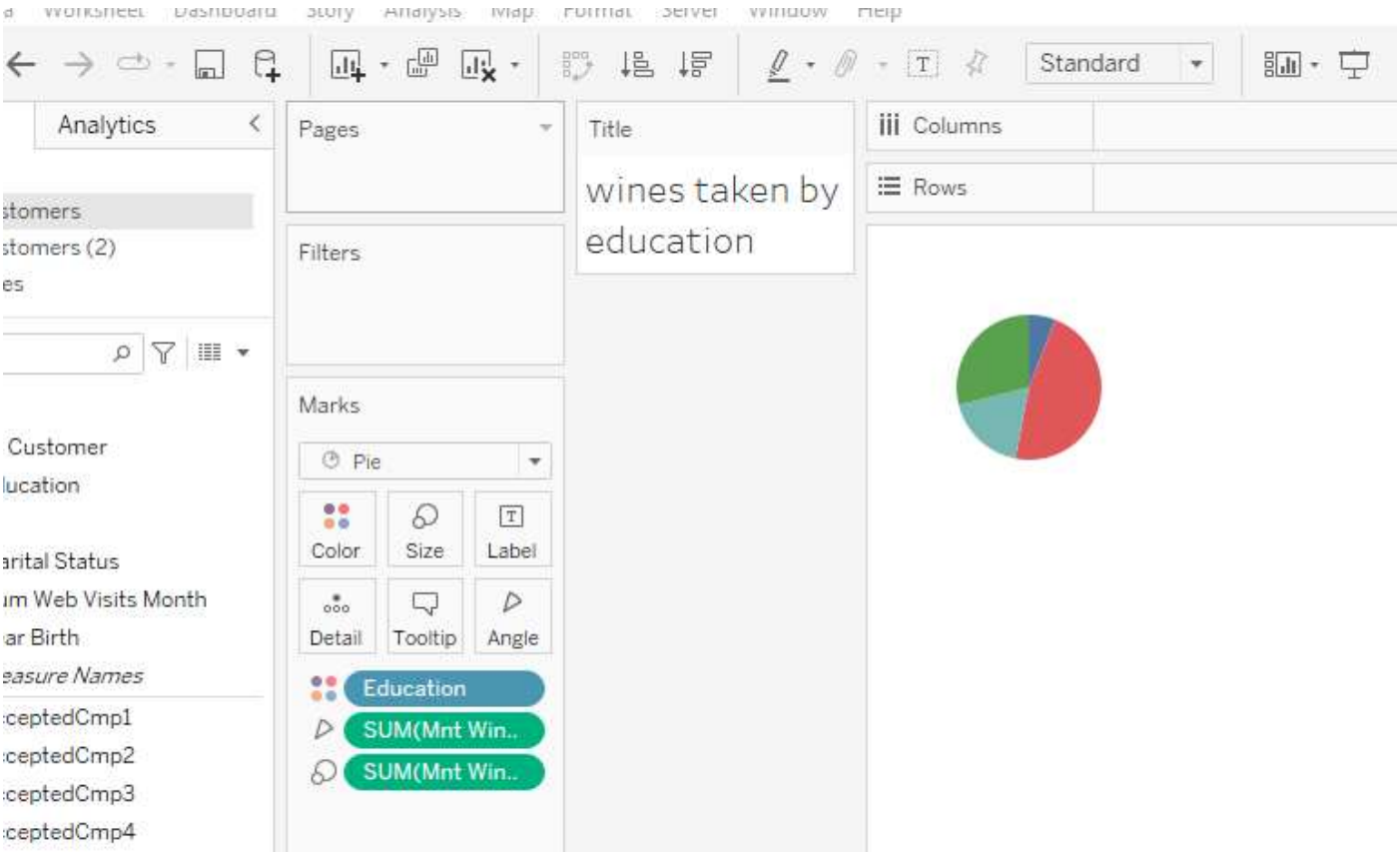


Figure 29 - According to Education

Analysing Data in Saless dataset using Tableu

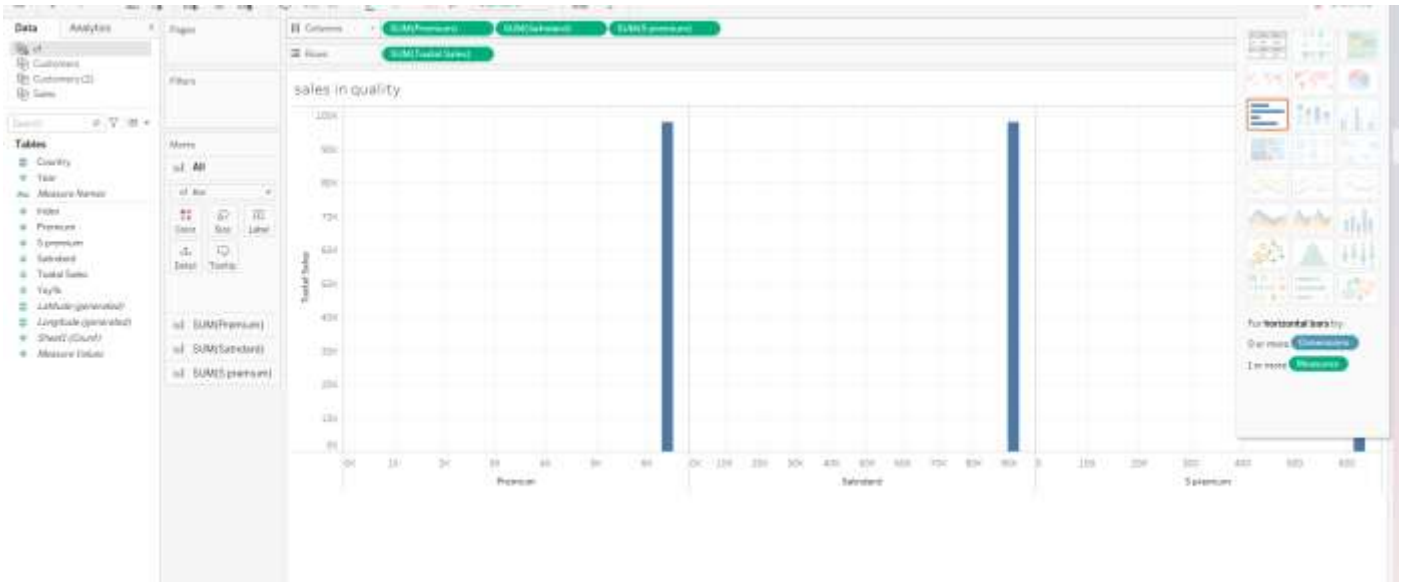


Figure 30- Sales according to quality

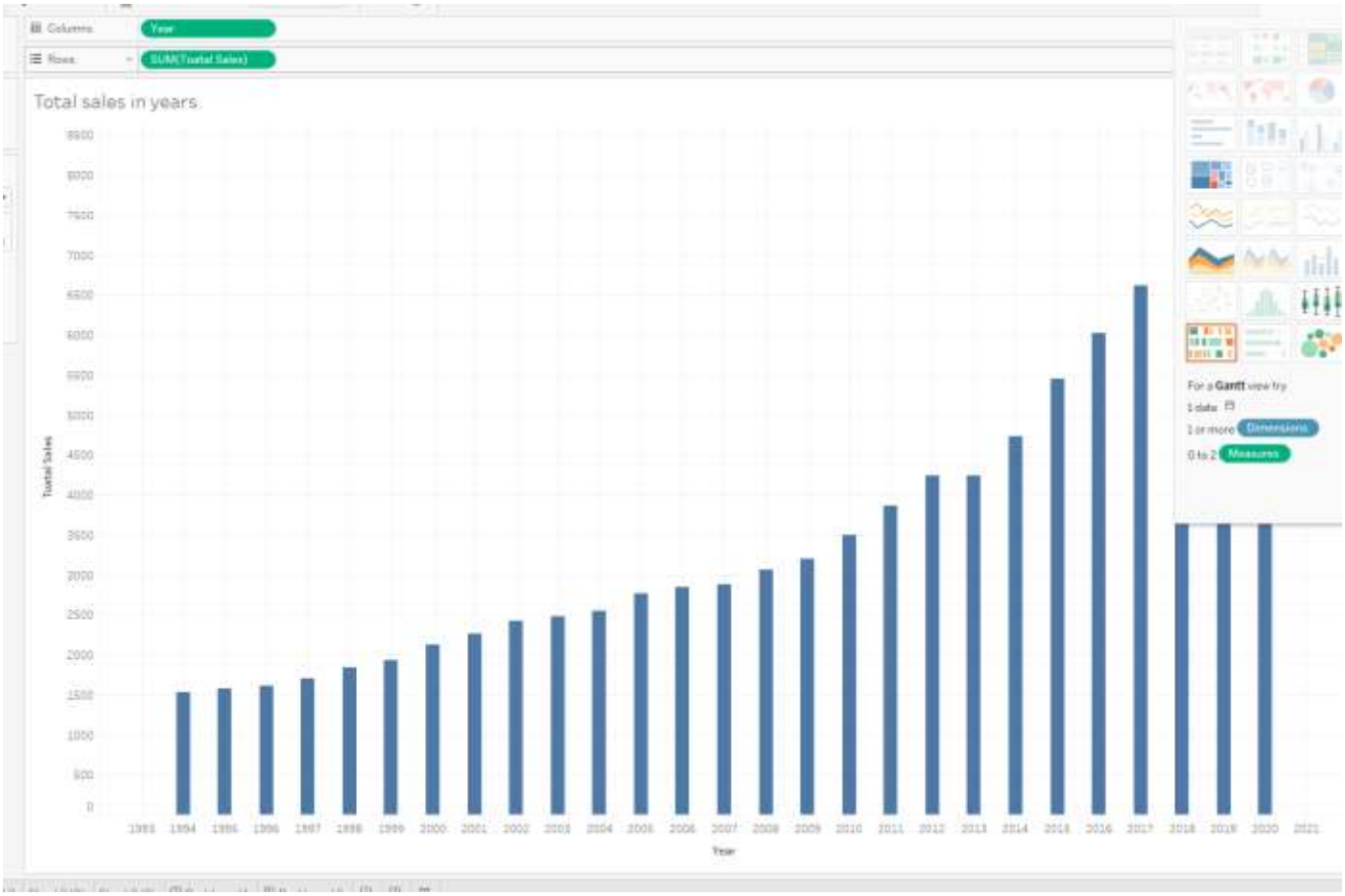
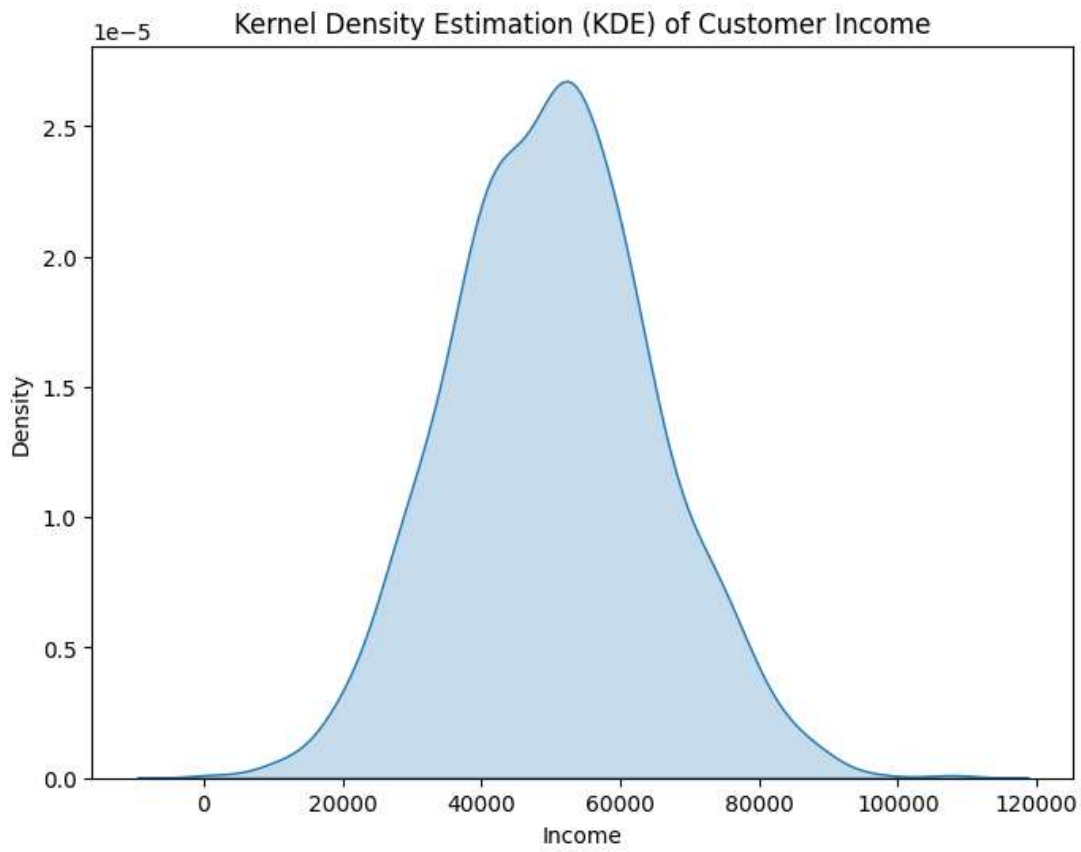


Figure 31 - Sales in Years

Figure 32 - Sales according to country

Creating a Kernel Density Estimation (KDE) plot to visualize the probability density function of customer income

```python
# KDE plot for customer income
plt.figure(figsize=(8, 6))
sns.kdeplot(df_customers['Income'], fill=True)
plt.title('Kernel Density Estimation (KDE) of Customer Income')
plt.xlabel('Income')
plt.ylabel('Density')
plt.show()
```



This code segment generates a Kernel Density Estimation (KDE) plot to visualize the probability density function of customer income in the synthetic dataset. The `plt.figure(figsize=(8, 6))` sets the size of the resulting plot, and the `sns.kdeplot` function from Seaborn is utilized to create the KDE plot. The 'Income' column from the `df_customers` DataFrame serves as the data source. The `fill=True` parameter is specified to fill the area under the KDE curve, enhancing visibility. Additional Matplotlib functions set the title ('Kernel Density Estimation (KDE) of Customer Income'), x-axis label ('Income'), and y-axis label ('Density'). This plot offers a smoothed representation of the distribution of customer income, providing insights into the likelihood of different income levels in the dataset. The KDE plot complements the histogram,

offering a continuous overview of the income distribution, which can be valuable for identifying trends and patterns in the data.

Applying a simple random sampling technique to select a subset of 100 customers from the synthetic dataset of customer income

```
# Simple random sampling of 100 customers
sample_size = 100
random_sample = df_customers['Income'].sample(sample_size, random_state=42)

# Display the first few rows of the random sample
random_sample.head()
```

```
521    58150.402886
737    64740.364759
740    22386.886530
660    41395.069897
411    33130.368622
Name: Income, dtype: float64
```

Figure 33 - simple random sampling technique to select a subset of 100 customers from the synthetic dataset of customer income

In this code snippet, a simple random sampling technique is applied to select a subset of 100 customers from the synthetic dataset of customer income. The variable `sample_size` is set to 100, indicating the desired size of the random sample. The `df_customers['Income'].sample` function is used to randomly extract 100 entries from the 'Income' column of the `df_customers` DataFrame. The parameter `random_state=42` ensures reproducibility, meaning that running this code with the same random state will yield the same random sample. Finally, the `head()` method is employed to display the first few rows of the randomly selected customer income sample. Simple random sampling is a fundamental method in statistics, providing an unbiased representation of the larger dataset and facilitating analyses on a more manageable subset of the data.

Introducing a categorical variable into the synthetic customer income dataset for illustrative purposes

```
# Create a categorical variable for demonstration purposes (replace with your actual categorical variable)
df_customers['Category'] = pd.cut(df_customers['Income'], bins=[0, 40000, 80000, np.inf], labels=['Low', 'Medium', 'High'])

# Stratified sampling based on the 'Category' variable
stratified_sample = df_customers.groupby('Category', group_keys=False).apply(lambda x: x.sample(min(len(x), 30), random_state=42))

# Display the first few rows of the stratified sample
stratified_sample.head()
```

	Income	Category
103	37965.840962	Low
26	32735.096339	Low
589	37158.742611	Low
966	39854.115469	Low
982	35495.357853	Low

Figure 34 - categorical variable into the synthetic customer income dataset for illustrative purposes

In this code segment, a categorical variable is introduced into the synthetic customer income dataset for illustrative purposes. The `pd.cut` function is utilized to create the categorical variable named 'Category' based on income ranges. The income values are divided into bins—'Low' for income up to 40,000, 'Medium' for income between 40,000 and 80,000, and 'High' for income above 80,000. This categorical variable allows for the demonstration of stratified sampling, a method that divides the dataset into distinct strata or groups based on the 'Category' variable. The `groupby` function is applied to the 'Category' column, and the `apply` method, along with a lambda function, is used to independently sample 30 entries from each

stratum. The `min(len(x), 30)` ensures that if a stratum has fewer than 30 entries, the entire stratum is included in the sample. The resulting DataFrame, 'stratified_sample,' showcases a stratified subset of the original data, where each income category is proportionally represented. Stratified sampling is particularly useful when ensuring that each subgroup is adequately represented in the final sample.

Defining Big Data and discussing its potential usefulness for Captain Flint Brewery in increasing sales.

Big data refers to a combination of structured, semistructured, and unstructured data amassed by organizations, presenting opportunities for extraction of valuable insights and utilization in advanced analytics applications, machine learning projects, and predictive modeling. The key attributes of big data, often referred to as the three V's—volume, variety, and velocity—highlight the substantial scale, diverse types, and rapid generation and processing speed of the data. Originally identified in 2001 by Doug Laney, these characteristics have become foundational in understanding big data. Additional V's, such as veracity, value, and variability, have been introduced to capture more dimensions of its complexity. While big data itself doesn't specify a particular volume, deployments frequently involve massive amounts of data, ranging from terabytes to exabytes, collected and accumulated over time.

For Captain Flint Brewery, the potential usefulness of big data in enhancing sales is significant. Firstly, the large volume of data allows the brewery to gather and analyze extensive information about customer preferences, purchasing behaviors, and market trends. By leveraging this data, the brewery can gain valuable insights into the demand for different alcoholic beverages and adjust its production and marketing strategies accordingly. Secondly, the wide variety of data

types in big data systems enables the brewery to capture diverse information, including customer demographics, social media interactions, and feedback. This comprehensive view aids in tailoring products and marketing campaigns to specific customer segments, fostering customer engagement and loyalty.

The velocity at which data is generated is crucial for real-time decision-making. With big data analytics, Captain Flint Brewery can swiftly analyze streaming data from various sources, allowing timely responses to market changes, competitor activities, and customer preferences. This agility is vital in a competitive industry, enabling the brewery to stay ahead of trends and adapt swiftly to dynamic market conditions. Additionally, considering the other V's, such as veracity (ensuring data accuracy and reliability) and value (extracting actionable insights), big data provides a robust foundation for making informed business decisions and optimizing operational efficiency.

Furthermore, variability in data sources, such as fluctuations in seasonal demand or changing consumer behaviors, is accommodated by big data analytics. Captain Flint Brewery can harness predictive modeling to anticipate shifts in consumer preferences, optimize inventory management, and plan targeted marketing campaigns. The ability to handle variability ensures that the brewery remains agile in adapting to market dynamics and optimizing its product offerings.

By considering the above, big data presents a transformative opportunity for Captain Flint Brewery to revolutionize its operations, marketing strategies, and overall sales performance. By tapping into the insights derived from large volumes and diverse types of data, the brewery can enhance its understanding of the market, improve customer relations, and make data-driven decisions that contribute to increased sales and sustained business growth.

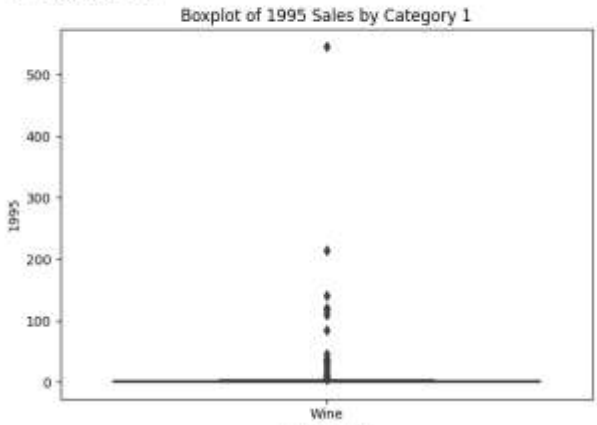
(Botelho, 2023)

Exploratory Data Analysis (EDA) and Visualizations for Sales Data in the Year 1995 creating a dashboard

```
[84] # Convert '1995' column to numeric
     df_sales['1995'] = pd.to_numeric(df_sales['1995'], errors='coerce')
```

```
# Creating a subplot
plt.subplot(2, 2, 1)
sns.histplot(df_sales['Category 1'])
plt.title('Distribution of Category 1')

plt.subplot(3, 1, 1)
sns.boxplot(x='Category 1', y='1995', data=df_sales)
plt.title('Boxplot of 1995 Sales by Category 1')

plt.tight_layout()
plt.show()
```

```
<ipython-input-88-61ddaca6c2af>:8: MatplotlibDeprecationWarning: Auto-removal of overlapping axes is deprecated since 3.6 and will be removed two minor releases later; explicitly call
  plt.subplot(1, 1, 1)
```



This code snippet is part of a data visualization process using the Seaborn library to analyze and present insights from a dataset, potentially related to big data. In the context of big data applications, companies often deal with massive volumes of data, and creating meaningful visualizations is crucial for understanding trends and making informed decisions.

The first line of code converts the '1995' column in the DataFrame `df_sales` to numeric format, ensuring that the data is suitable for numerical operations and visualizations. This step is essential, especially when dealing with large datasets where data types need to be appropriately managed.

The subsequent code block involves creating a subplot layout with two subplots. The first subplot (2, 2, 1) displays a histogram of the 'Category 1' column, providing a visual representation of the distribution of data in that category. Understanding the distribution is fundamental in big data analytics as it can reveal patterns and anomalies within the dataset.

The second subplot (1, 1, 1) presents a boxplot depicting the sales data for the year 1995 categorized by 'Category 1.' Boxplots are valuable in big data analytics as they provide insights into the distribution of data, highlighting potential outliers and the central tendency of sales within each category.

The combination of histograms and boxplots in a dashboard can offer a comprehensive view of the dataset, facilitating the identification of trends and outliers within big data. The `plt.tight_layout()` ensures that the subplots are appropriately arranged, and `plt.show()` displays the final dashboard.

In the broader context of big data applications, such visualizations aid decision-makers in extracting actionable insights from vast datasets, potentially leading to improved strategies and outcomes. The ability to create informative dashboards becomes particularly significant when dealing with the complexities and scale associated with big data analytics.

Dashboards made for Sales and Customer datasets using tableu public

Dashboard for sales data

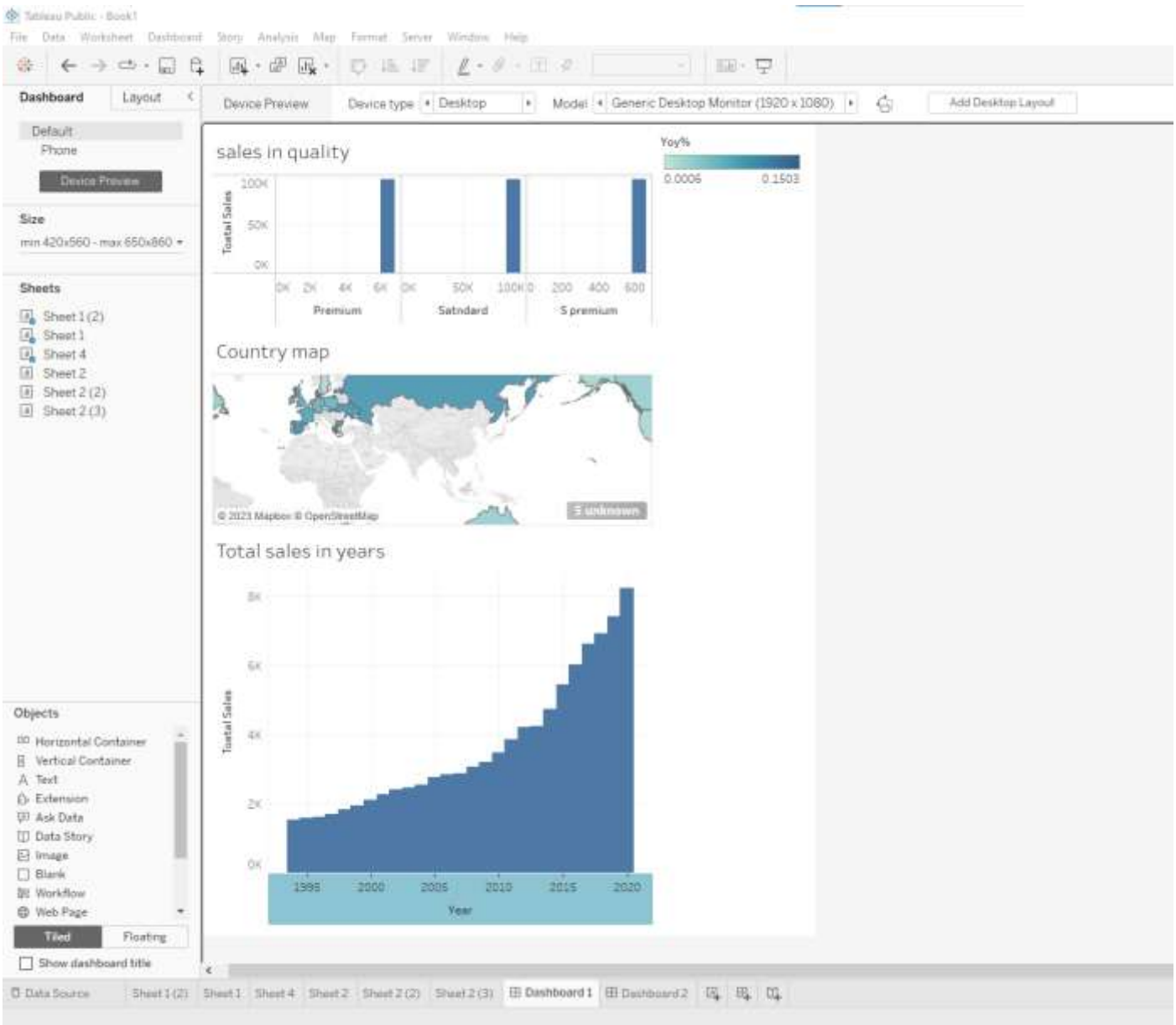


Figure 35 - Dashboard for sales data

Here a dashboard created using tableu can be displayed by collaborating the sheets of sales in quantity

Country maps and total sales in years.

Dashboard for customer data

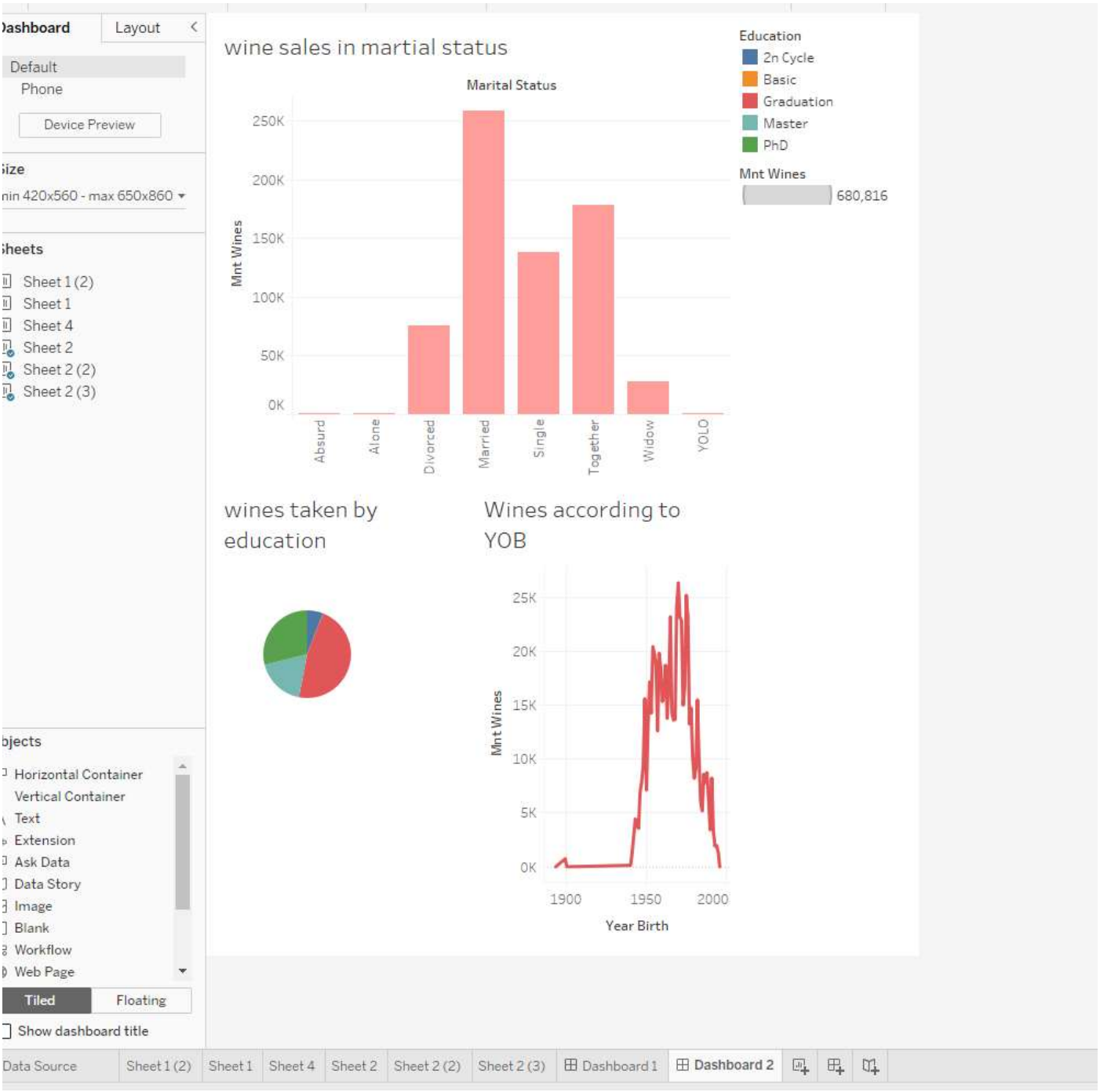


Figure 36 - Dashboard for customer data

Here a dashboard created using tableu can be displayed by collaborating the sheets of wine sales as martial , wines taken in education and wines taken according to YOB

Applying an appropriate tool to demonstrate these predictive analytic techniques.

Tableau Public has limitations compared to the full Tableau Desktop version, you can still perform various predictive analytics tasks using its features. Below are steps for some of the predictive analytics techniques mentioned earlier using Tableau Public:

Time series Forecasting

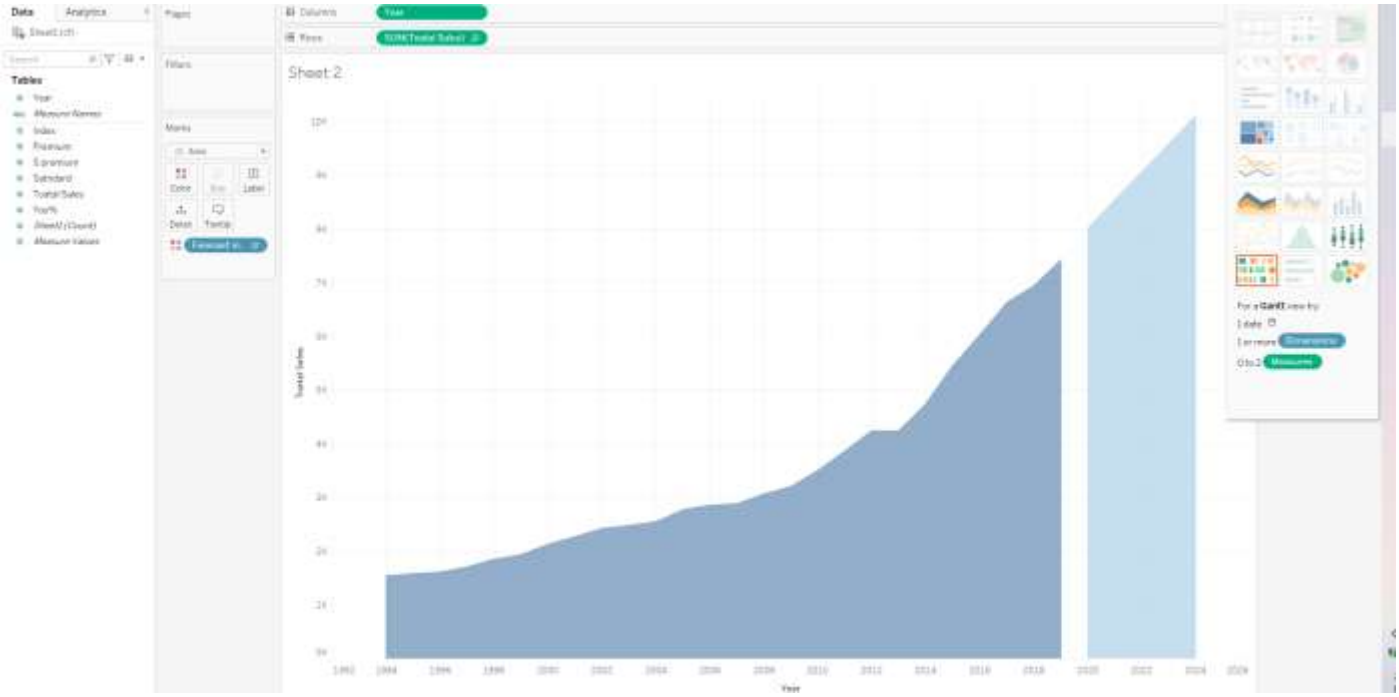


Figure 37 - Time series Forecasting

To conduct time series forecasting in Tableau Public for total sales over the years, begin by loading your dataset, ensuring it contains a time-related variable (e.g., date) and the metric to

predict (e.g., total sales). Create a time series chart by dragging the time-related variable to the "Columns" shelf and the sales metric to the "Rows" shelf. Tableau will automatically generate a time series chart depicting total sales over the years. Enable forecasting by right-clicking on the chart and selecting "Forecast." This activates Tableau's predictive capabilities, extending the chart to show forecasted values. Explore the forecasted trends by hovering over the projected portion of the chart, gaining insights into potential future outcomes based on historical sales data. This visual approach in Tableau aids in understanding sales patterns and making informed decisions through time-based predictions.

Regression Analysis

Figure 38 - Regression Analysis

For conducting regression analysis on total sales and sum sales in Tableau Public, start by importing your dataset into the platform, ensuring it includes the necessary variables for regression analysis. Create a scatter plot by dragging the independent variable, such as total sales, to the "Columns" shelf, and the dependent variable, like sum sales, to the "Rows" shelf.

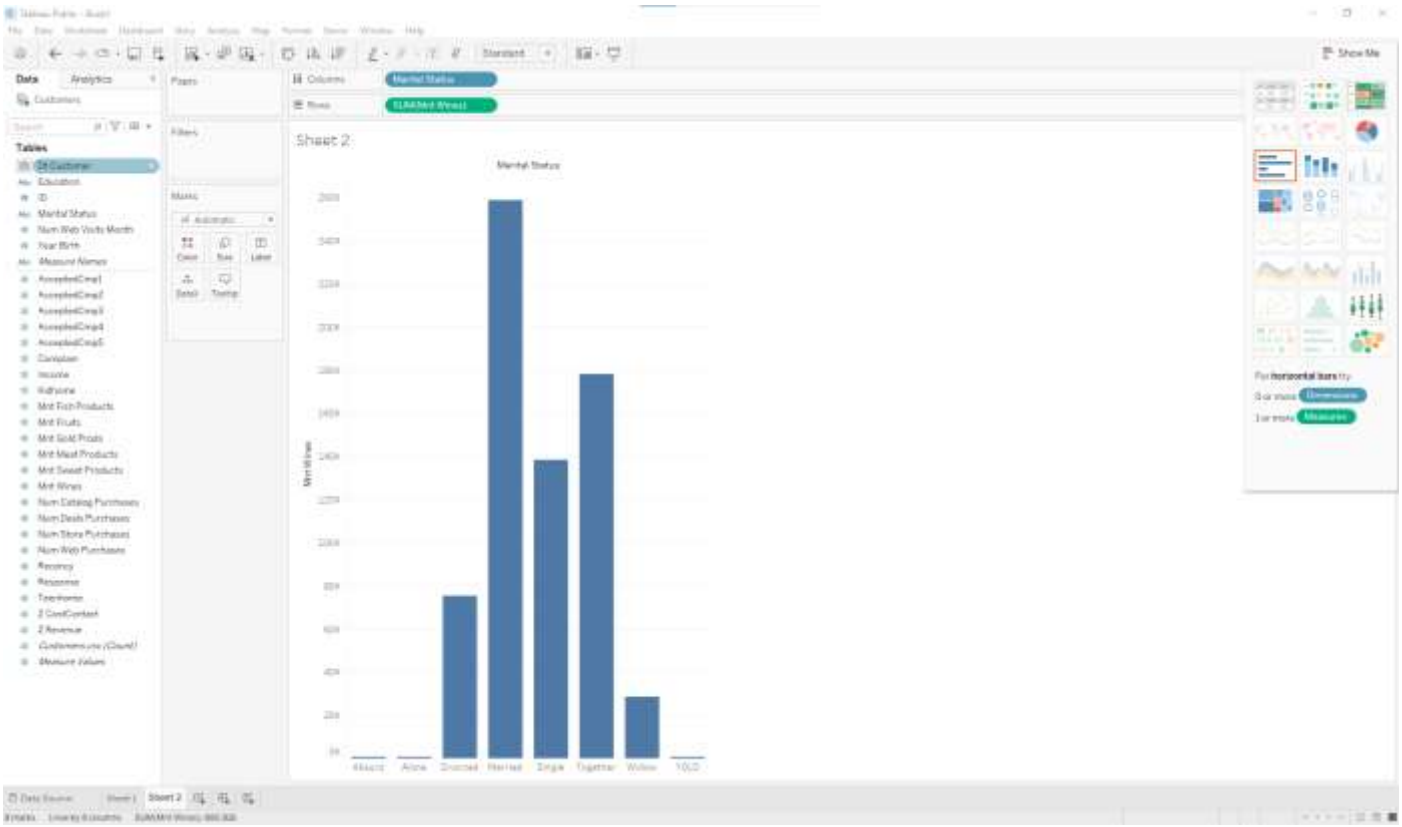
Tableau will automatically generate a scatter plot illustrating the relationship between the two variables. To fit a regression model, right-click on the plot, go to "Trend Lines," and select the appropriate regression model, whether it's linear or polynomial. This action introduces a regression line or curve to the scatter plot, visually representing the modeled relationship between total sales and sum sales. Exploring this regression analysis in Tableau provides valuable insights into how changes in one variable may impact another, aiding in predictive modeling and decision-making processes related to sales.

Classification Models

For implementing classification models on Tableau Public, initiate the process by importing your dataset, ensuring it contains the necessary variables for classification analysis. Depending on the nature of data, create a scatter plot or bar chart with the relevant variables to visually explore patterns. Although Tableau Public may have limitations in constructing intricate classification models, the user can leverage its visual analytics capabilities to identify trends that might indicate potential classification patterns. While Tableau may not directly support decision trees or logistic regression within its interface, using visual cues from the charts can guide your understanding of the data's classification characteristics. This approach allows the user to gain insights into the relationships between variables and potential predictive patterns, contributing to a preliminary exploration of classification tendencies in your dataset.

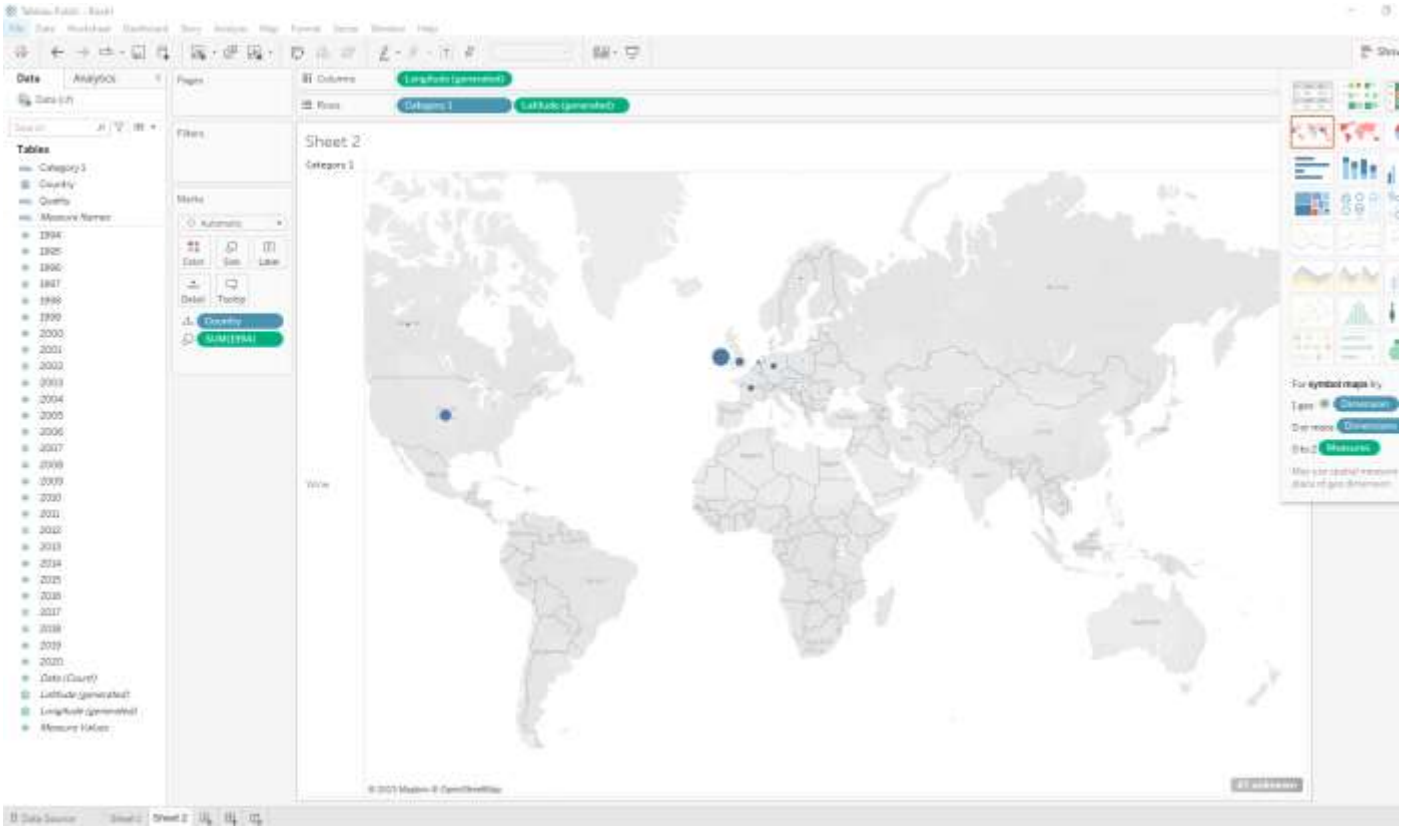
Other visualizations

Custom



In Tableau, the visualization process involves an intuitive drag-and-drop interface where users can seamlessly place variables onto Rows and Columns shelves to construct visualizations. The flexibility of Tableau enables users to apply filters, refining their focus on specific data points or time ranges for a more detailed analysis. Custom calculations can be implemented by creating calculated fields, allowing for additional customization or transformations tailored to specific analytical needs. Once the analysis is complete and the desired visualizations are generated, users can easily share their insights by publishing the workbook to Tableau Public. This platform serves as a hub for sharing interactive and visually appealing data visualizations, enabling collaboration and knowledge dissemination among users. These general steps encapsulate the user-friendly and dynamic nature of Tableau for creating impactful visualizations from diverse datasets.

In a map



Once imported, drag the geographical field representing countries to the "Rows" shelf and Tableau will automatically recognize it as a geographic dimension. Simultaneously, drag the metric variable you want to visualize (e.g., sales, population) to the "Color" or "Size" shelf to represent the distribution. Tableau will create a map visualization where each country is color-coded or sized based on the chosen metric, allowing for a clear representation of the distribution across different regions. Adjust the map settings, add labels, and customize the appearance to enhance the clarity of your geographical distribution visualization in Tableau. This approach enables users to quickly interpret and gain insights into the spatial distribution of data across various countries within the dataset.

Using python to forecast for 5 years using time series in python for sales dataset

```
[12] cfdata = df_cf[['Index', 'Year', 'Toatal Sales']].rename(columns={'Year': 'ds', 'Toatal Sales': 'y'})
```

```
[13] cfdata.set_index('ds', inplace=True)
```

```
[14] cfdata['y'] = pd.to_numeric(cfdata['y'].str.replace(',', ''), errors='coerce')
```

```
[15]
     model = ExponentialSmoothing(cfdata['y'], seasonal='add', seasonal_periods=3)
     fit_model = model.fit()
```

```
/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: An unsupported index was provided and will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/holtwinters/model.py:917: ConvergenceWarning: Optimization failed to converge. Check mle_retvals.
  warnings.warn(
```

```
[18] future_years = np.arange(cfdata.index.max() + 1, cfdata.index.max() + 6)
     future = pd.DataFrame({'ds': future_years})
     forecast = fit_model.forecast(steps=5)
```

```
/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/base/tsa_model.py:836: ValueWarning: No supported index is available. Prediction results will be given with an integer index beg
  return get_prediction_index(
/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/base/tsa_model.py:836: FutureWarning: No supported index is available. In the next version, calling this method in a model witho
  return get_prediction_index(
```

```
# Plot the original data and the forecast
plt.figure(figsize=(12, 6))
plt.plot(cfdata.index, cfdata['y'], label='Actual Sales')
plt.plot(forecast.index, forecast, label='Forecasted Sales', linestyle='--')
plt.title('Total Sales Forecast for the Next 5 Years')
plt.xlabel('Year')
plt.ylabel('Total Sales')
plt.legend()
plt.show()
```

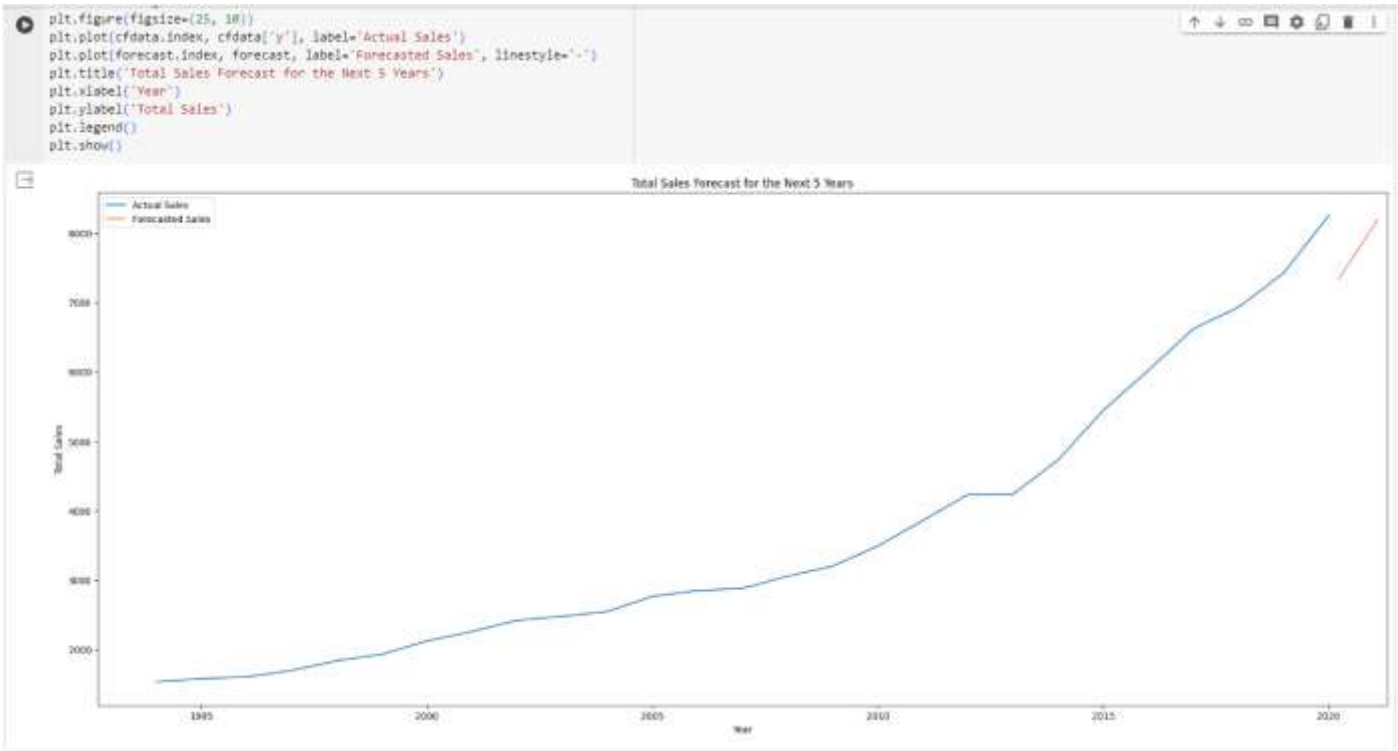


Figure 39 - Forecasting using time series

This Python code uses the 'statsmodels' library to perform time series forecasting of total sales over the years. It starts by preparing the sales data, which includes the 'Year' and 'Total Sales' columns, renaming them as 'ds' and 'y' respectively, and converting the 'y' values to numeric format. The code then applies the Exponential Smoothing model from 'statsmodels' to capture any underlying patterns in the sales data. Subsequently, it forecasts the total sales for the next 5 years and plots both the actual sales data and the forecasted values using a line graph. This visualization helps in understanding the predicted trend in total sales over the specified time frame, aiding in business planning and decision-making.’

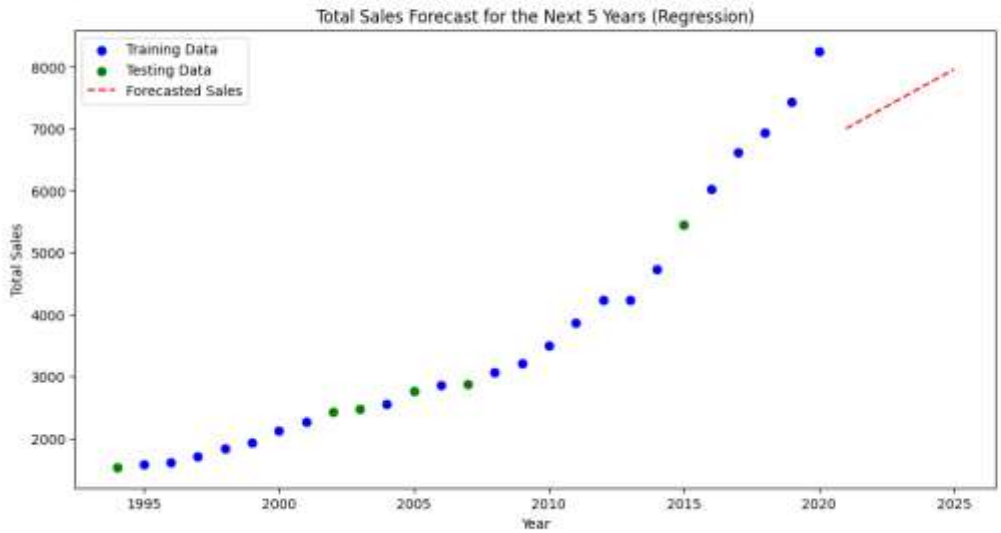
Using python to forecast for 5 years using regression in python for sales dataset

```
[21] from google.colab import files
     uploaded = files.upload()

     Choose Files  cf.csv
     • cf.csv(text/csv) - 1570 bytes, last modified: 12/27/2023 - 100% done
     Saving cf.csv to cf.csv

[38] cfdata = pd.read_csv('cf.csv')

[32] cfdata['Total Sales'] = pd.to_numeric(cfdata['Toatal Sales'].str.replace(',', ''), errors='coerce')

[39] model = make_pipeline(
         SimpleImputer(strategy='mean'),  # You can choose 'mean', 'median', or 'most_frequent'
         LinearRegression()
     )

[42] cfdata.dropna(subset=['Year', 'Toatal Sales'], inplace=True)

[ ] Start coding or generate with AI.

[34] X_train, X_test, y_train, y_test = train_test_split(cfdata[['Year']], cfdata['Total Sales'], test_size=0.2, random_state=42)

[35] reg_model = LinearRegression()

[43] X_train, X_test, y_train, y_test = train_test_split(cfdata[['Year']], cfdata['Toatal Sales'], test_size=0.1, random_state=42)

[45]
     # Convert 'Toatal Sales' to numeric
     y_train = pd.to_numeric(y_train.str.replace(',', ''), errors='coerce')
     y_test = pd.to_numeric(y_test.str.replace(',', ''), errors='coerce')

[46] model.fit(X_train, y_train)

     Pipeline
```

Figure 40 - Forecasting using regression

```
[46] model.fit(X_train, y_train)

     Pipeline
     SimpleImputer
     LinearRegression

[47] # Predict sales for the next 5 years
     future_years = np.arange(cfdata['Year'].max() + 1, cfdata['Year'].max() + 6)
     future = pd.DataFrame({'Year': future_years})

     forecast = model.predict(future[['Year']])

     # Plot the original data and the forecast
     plt.figure(figsize=(12, 6))
     plt.scatter(X_train, y_train, label='Training Data', color='blue')
     plt.scatter(X_test, y_test, label='Testing Data', color='green')
     plt.plot(future, forecast, label='Forecasted Sales', linestyle='--', color='red')
     plt.title('Total Sales Forecast for the Next 5 Years (Regression)')
     plt.xlabel('Year')
     plt.ylabel('Total Sales')
     plt.legend()
     plt.show()
```

```
# Plot the original data and the forecast
plt.figure(figsize=(12, 8))
plt.scatter(X_train, y_train, label='Training Data', color='blue')
plt.scatter(X_test, y_test, label='Testing Data', color='green')
plt.plot(future, forecast, label='Forecasted Sales', linestyle='--', color='red')
plt.title('Total Sales Forecast for the Next 5 Years (Regression)')
plt.xlabel('Year')
plt.ylabel('Total Sales')
plt.legend()
plt.show()
```



Double-click (or enter) to edit

This Python code utilizes a linear regression model to forecast total sales for the next 5 years based on historical data stored in a DataFrame named 'cfdata.' The initial step involves cleaning the 'Toatal Sales' column, addressing issues such as commas in numeric representations. A pipeline is then created using scikit-learn, incorporating a mean imputer to handle any missing values and a linear regression model for prediction. Rows with missing values in the 'Year' and 'Toatal Sales' columns are dropped, and the data is split into training and testing sets. The 'Toatal Sales' column is converted to numeric format to ensure compatibility with the linear regression model. Subsequently, the model is fitted on the training data, and sales are predicted for the next 5 years. The forecasted values are plotted alongside the original training and testing data using matplotlib, providing a visual representation of the predicted sales trends over time.

It's important to note that this code assumes the presence of a DataFrame named 'cfdata' with specific column names, and adjustments may be needed based on the actual structure of your

dataset. Additionally, considerations such as handling missing values and ensuring proper data type conversions are crucial for accurate model training and forecasting.

Evaluation of how predictive analytic techniques can be used for forecasting purposes

Predictive analytics is an advanced analytical approach that uses historical data, statistical modeling, data mining techniques, and machine learning to make predictions about future outcomes. It is widely applied in various industries to identify patterns, assess risks, and uncover opportunities by analyzing past data. Predictive analytics is closely associated with big data and data science, leveraging large datasets to enhance its forecasting capabilities.

Predictive analytics techniques play a crucial role in forecasting by using historical data and advanced algorithms to make predictions about future outcomes. These techniques analyse patterns and relationships within the data, identifying trends that can help businesses anticipate future events. For example, in retail, predictive analytics can be applied to forecast sales for upcoming seasons by considering past sales patterns, customer behaviours, and external factors like economic conditions. By leveraging these insights, businesses can make informed decisions, such as adjusting inventory levels or planning targeted marketing campaigns to meet anticipated demand.

One key advantage of predictive analytics in forecasting is its ability to go beyond simple extrapolation from the past. Instead of just looking at historical trends, predictive models consider a range of variables and their potential impact on future outcomes. This approach allows businesses to adapt to changing conditions, mitigate risks, and capitalize on opportunities. Whether it's predicting customer preferences, demand for products, or financial performance, predictive analytics empowers organizations to make proactive and strategic decisions, ultimately improving their overall planning and operational efficiency.

There are different types of predictive modeling techniques, each serving specific purposes. Classification models, falling under supervised machine learning, categorize data based on

historical patterns. Examples include logistic regression, decision trees, random forest, neural networks, and Naïve Bayes. These models are useful for tasks like fraud detection and credit risk evaluation.

Clustering models, falling under unsupervised learning, group data based on similar attributes. Common algorithms include k-means clustering, mean-shift clustering, DBSCAN, EM clustering using GMM, and hierarchical clustering. Clustering helps identify groups of similar entities, allowing businesses to tailor strategies for each group.

Time series models focus on data with a time component, like daily, weekly, or monthly intervals. Autoregressive (AR), moving average (MA), ARMA, and ARIMA models are commonly used for time series analysis. For instance, a call center can use a time series model to forecast hourly call volumes.

In various industries, predictive analytics has proven invaluable. In banking, it aids in predicting loan defaults and identifying high-risk customers. In healthcare, it is used for patient care management and infection tracking. Human resources leverage predictive analytics for hiring and employee retention, while marketing and sales teams use it for customer engagement strategies. Supply chain management benefits from predictive analytics for inventory and pricing strategies.

Comparing predictive analytics to forecasting, predictive analytics is not mere guesswork but employs advanced algorithms, including machine learning and AI. Forecasting relies on past data to estimate future trends, whereas predictive analytics uses current and historical data to create specific models for identifying patterns and possible outcomes.

In business planning, forecasting estimates trends based on past data, like predicting sales margins for seasonal products. On the other hand, predictive analytics identifies potential customers, allowing businesses to understand target audiences, tailor marketing strategies, and optimize supply chains based on customer behavior.

Forecasting provides macro-level insights, helping businesses strategize for market challenges and opportunities. Predictive analytics, offering micro-level insights, dives into individual customer preferences, aiding in delivering personalized experiences for maximum satisfaction.

Ultimately, leveraging both forecasting and predictive analytics enhances decision-making in business operations. It is not a matter of choosing one over the other but understanding how to use each intelligently for different contexts. In the competitive market, employing both systematically and appropriately ensures better results and growth for businesses.

(Selerity, 2023)

(IBM, 2023)

Using a suitable prescriptive analytics method on the given business case of EMC.

To apply prescriptive analytics to the business case of EMC using Tableau, we start by identifying the specific goals or decision points that need optimization. Let's consider an example where EMC aims to enhance its supply chain operations. In Tableau, we gather relevant data about past sales, customer behaviors, and operational efficiency. This data is then processed and cleaned to ensure accuracy.

Next, we create a prescriptive model using Tableau's capabilities, utilizing optimization techniques such as linear programming. The model is designed to recommend the most efficient allocation of resources to minimize costs within the supply chain. To ensure its effectiveness, we test the model against a validation dataset and make any necessary adjustments based on the results.

Once the prescriptive model is validated, we leverage Tableau for optimization and simulation. This involves running different scenarios to understand the impact of various decisions, like adjusting resource allocations or modifying pricing strategies. Tableau's visualization tools help us interpret and communicate these insights effectively.

As we implement the recommendations derived from the prescriptive model into EMC's decision-making process, Tableau aids in visualizing the data-driven insights. For instance, if the model suggests reallocating resources in a certain way, Tableau's visualizations can help stakeholders comprehend the implications and benefits.

Continuous monitoring of the implemented decisions is crucial. Tableau allows us to create dashboards that track ongoing results against expected outcomes. Regular reviews enable us to refine the prescriptive model based on real-world feedback and any changes in the business environment.

Throughout this process, collaboration with EMC's domain experts and stakeholders remains essential. Tableau serves as a user-friendly platform, facilitating effective communication and

alignment between data analysts and business decision-makers. By leveraging Tableau's capabilities, EMC can optimize its supply chain operations and make informed decisions to enhance overall efficiency.

Data exploration

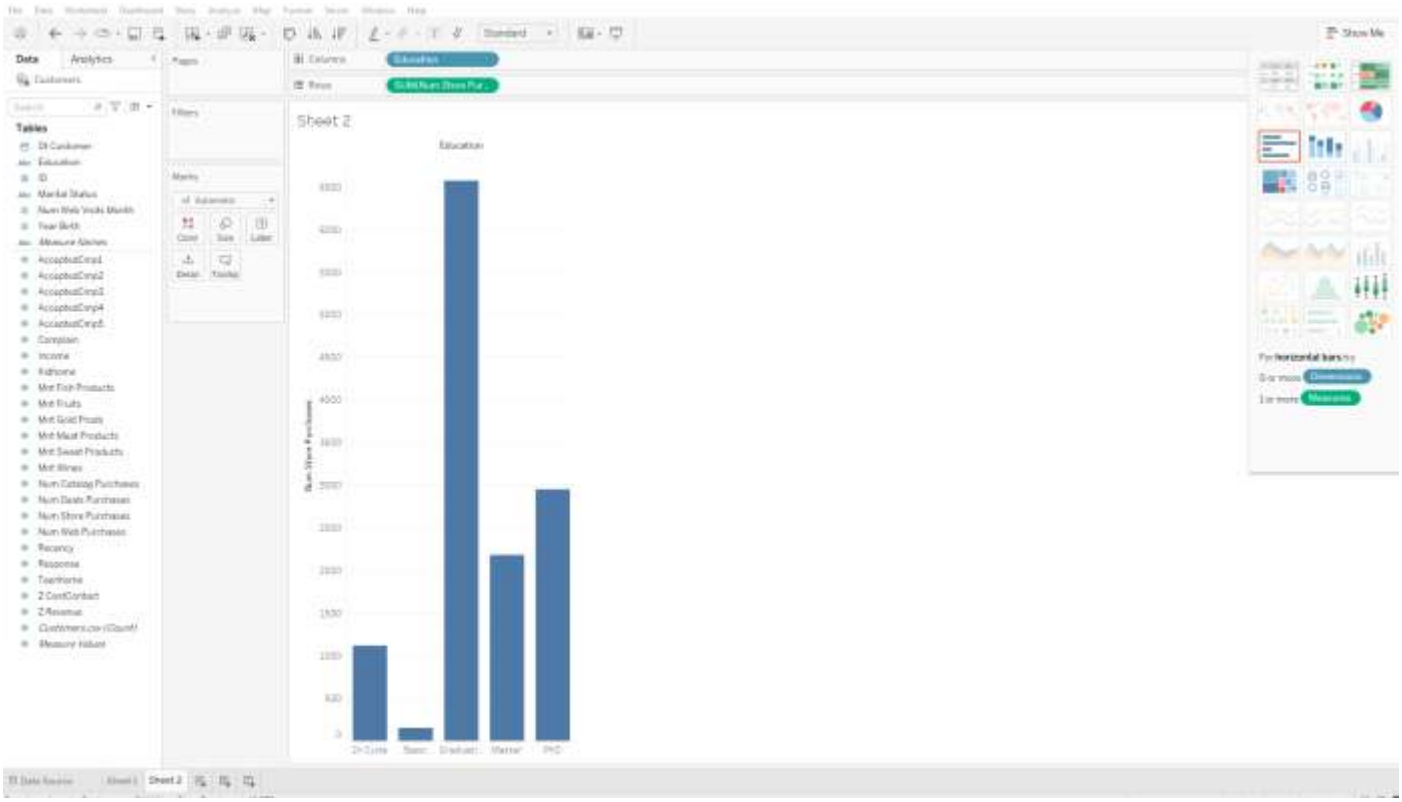


Figure 41 -Explore the dataset in Tableau to understand its structure and contents.

Customer Segmentation

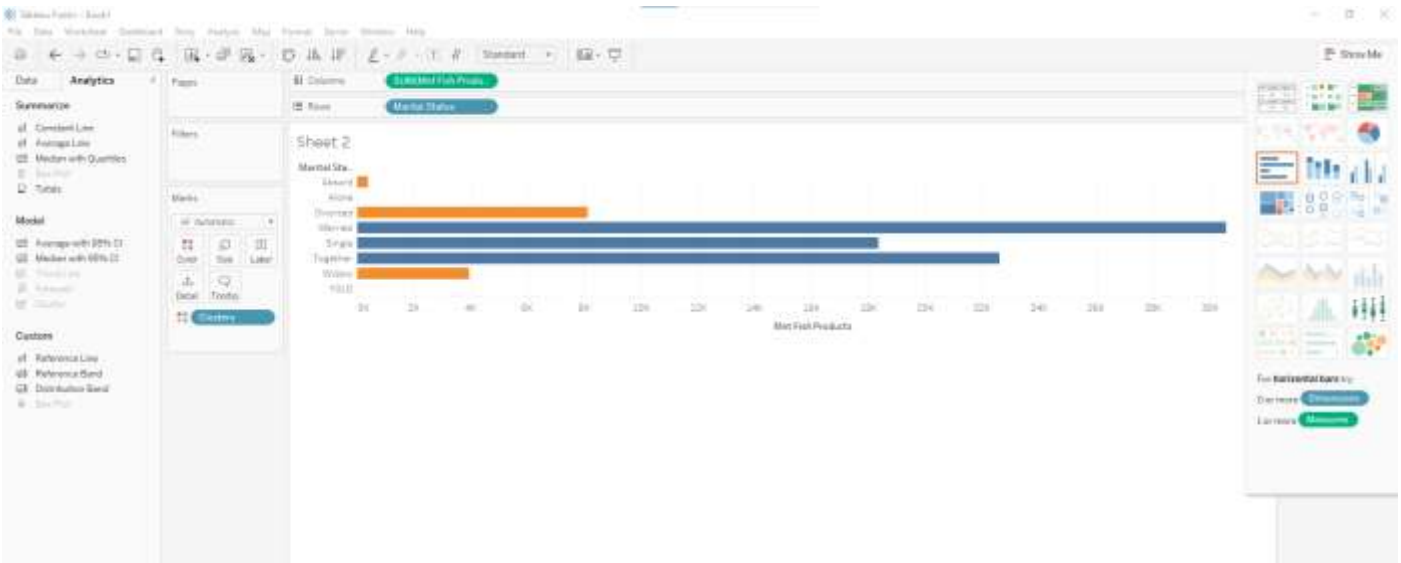
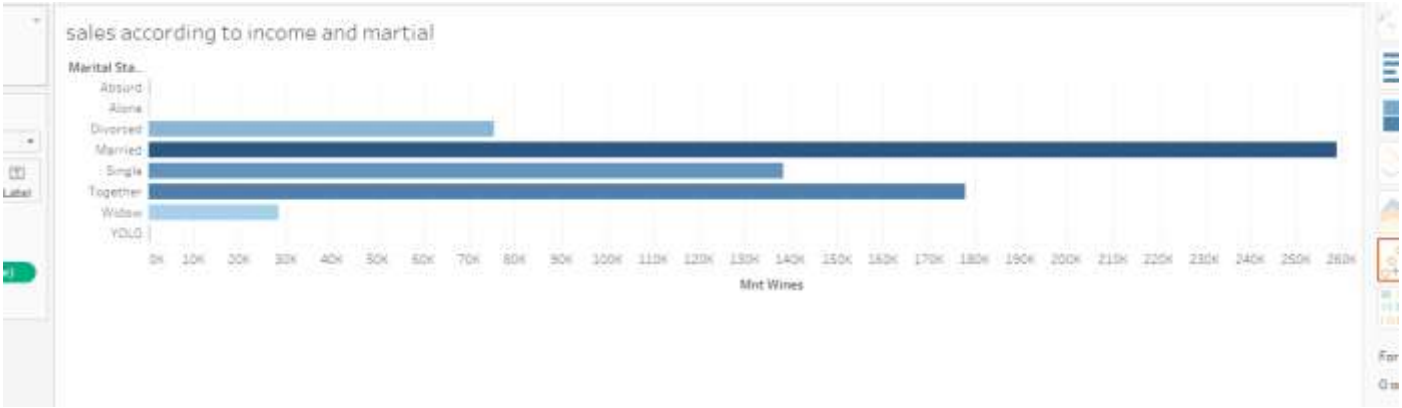


Figure 42 - use clustering algorithms, filters, or calculated fields to group customers into distinct types.



In the above figure we can consider that the organization can use these kind of data for prescriptive analysis and consider what kind of promotion strategies they can utilize in order to increase their goals.

Wine spending analysis

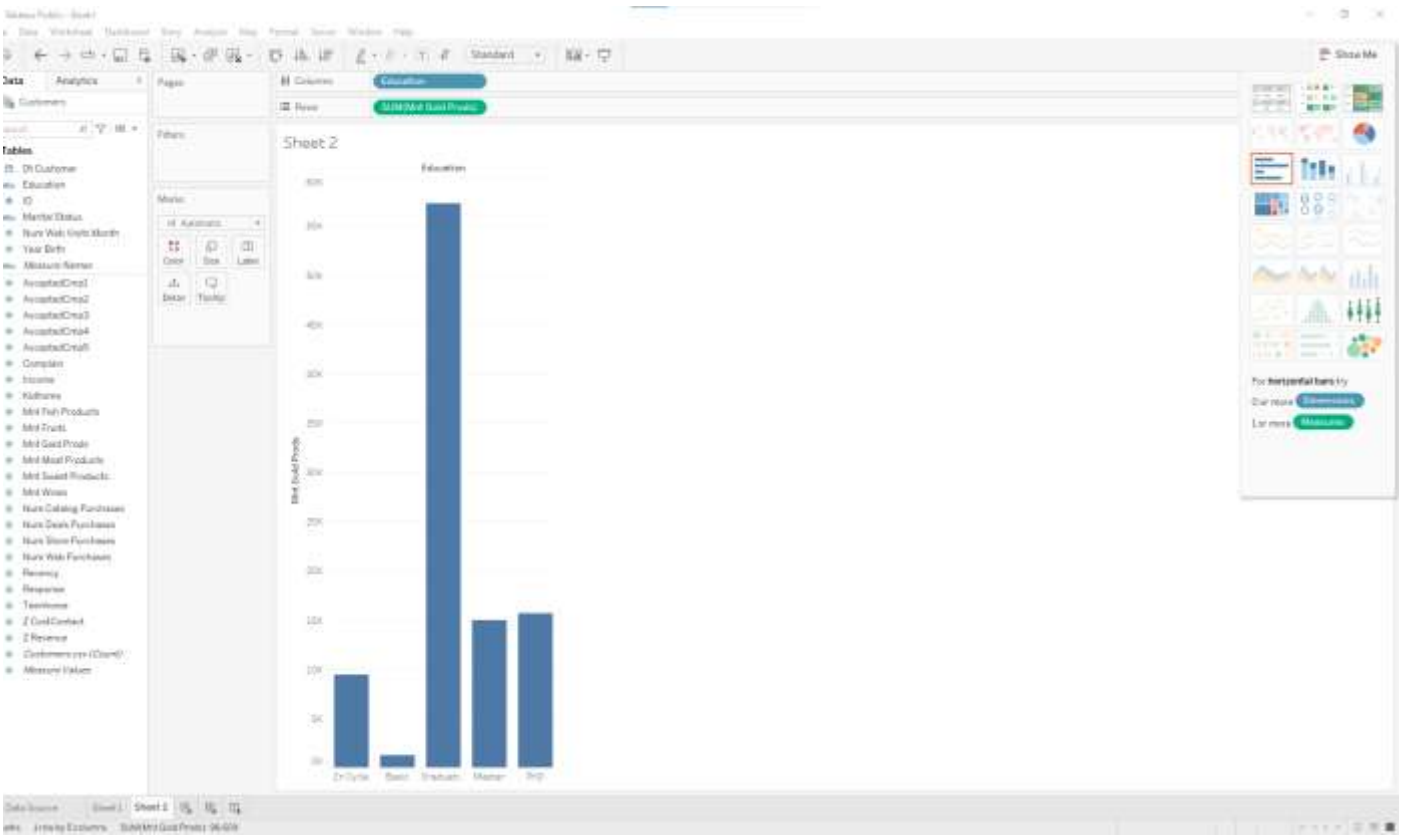


Figure 43 - Use charts like bar graphs or line charts to represent the distribution of spending across different customer segments.

Dashboards

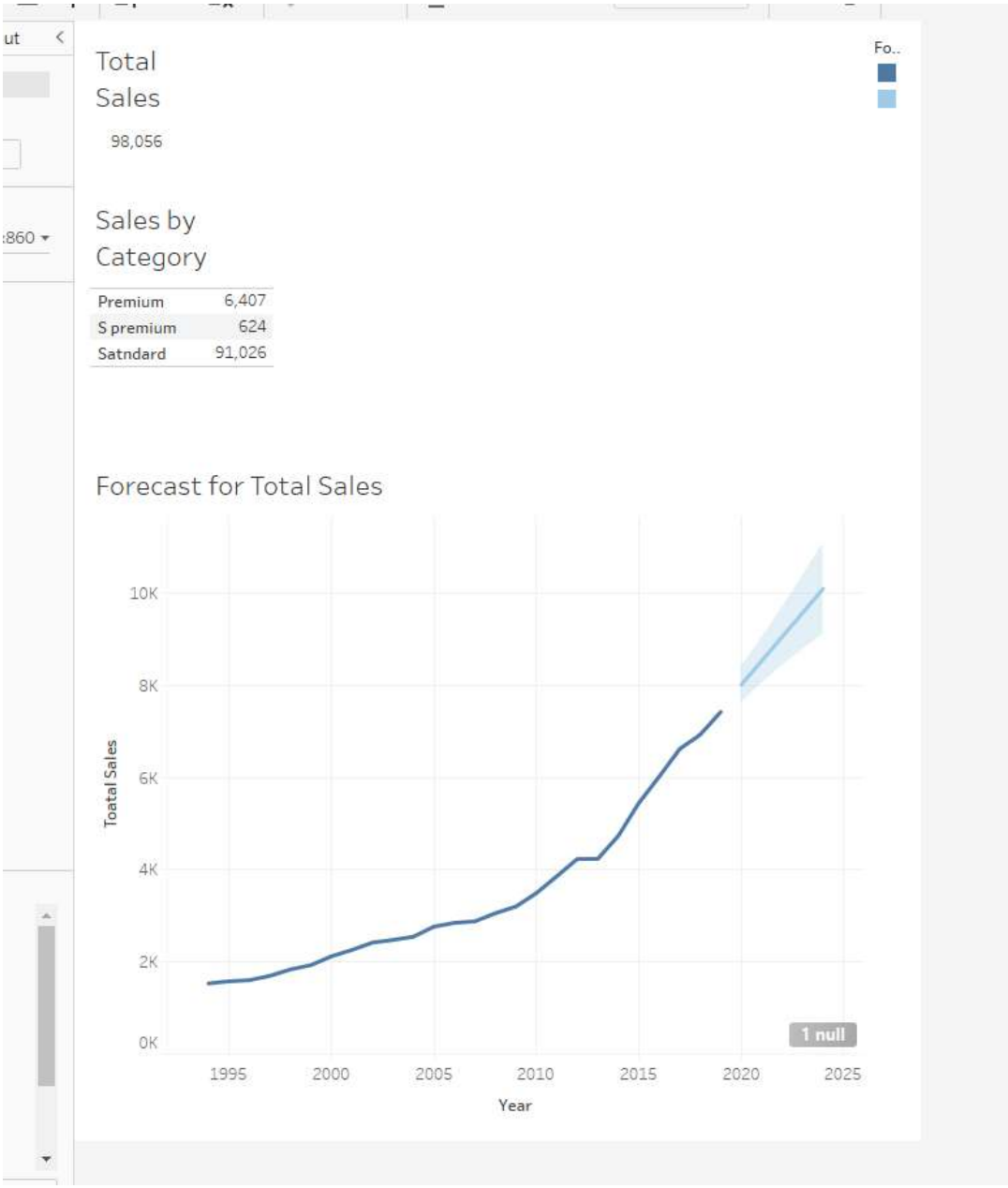


Figure 44 - Develop interactive dashboards in Tableau

Demonstrating these techniques using an appropriate programming language or tool.

Prescriptive analytics, a robust approach surpassing predictive insights, leverages tools like Tableau to guide optimal decision-making. The process begins with importing pertinent data into Tableau, encompassing historical and real-time information from diverse sources. Once collected, Tableau aids in data processing, rectifying missing or inaccurate data through normalization and pre-processing.

Identifying influential variables for decision-making is crucial, and Tableau allows for feature selection and engineering, empowering users to enhance predictive features. The subsequent step involves model creation, where Tableau supports various statistical, machine learning, and AI techniques. Testing and validating the model against a dataset in Tableau refine its performance, ensuring accurate predictions.

Tableau's capabilities extend to optimization and simulation, enabling the application of prescriptive models for scenario analysis. The outcomes are then utilized for informed decision-making, covering aspects from pricing strategies to resource allocation. Post-decision, Tableau facilitates continuous monitoring and review, forming a feedback loop for ongoing model refinement.

Considering the prescriptive analytics model in Tableau to align with specific business goals, such as optimizing costs or maximizing profits, is pivotal. The visualization prowess of Tableau comes to the forefront in representing decision impacts through interactive dashboards. Easy scenario analysis, facilitated by Tableau, allows for real-time evaluation of different decisions by adjusting variables.

Dynamic dashboards developed in Tableau serve as user-friendly interfaces for stakeholders, fostering collaboration and knowledge sharing. Sensitivity analysis, using Tableau, evaluates how changes in input variables influence the prescriptive model's output, identifying critical factors impacting decision outcomes.

The collaborative nature of Tableau promotes seamless sharing and dissemination of insights among team members and stakeholders. Lastly, Tableau contributes to continuous

improvement by gathering feedback and insights from ongoing operations, ensuring the prescriptive analytics model evolves for enhanced decision-making over time.

In summary, Tableau stands as an indispensable tool for implementing and showcasing prescriptive analytics techniques, guiding users through data collection, processing, model creation, decision-making, and continuous improvement in a cohesive and impactful manner.

Describing how these prescriptive analytic techniques are used to the best course of action in a situation.

Prescriptive analytics techniques play a crucial role in determining the best course of action in various situations. These methods go beyond predicting what might happen or why it happened; they provide recommendations on what actions to take to achieve desired outcomes.

Firstly, the process begins with data collection, where relevant information is gathered, including historical and real-time data. This data serves as the foundation for making informed decisions. Once collected, the data undergoes processing and cleaning to ensure accuracy, and features are selected or engineered to identify key variables.

After this, prescriptive models are created using techniques such as optimization, simulation, and machine learning. These models consider multiple scenarios and potential actions, helping to determine the optimal solution given specific constraints and objectives. For instance, a business might use prescriptive analytics to optimize its supply chain, finding the best balance between inventory levels and customer demand.

The models are then tested and validated to ensure their accuracy and effectiveness. This step is crucial as it allows decision-makers to have confidence in the recommendations provided by the prescriptive analytics system. Tableau, as a powerful tool, can be employed at various stages of this process, facilitating data visualization and scenario analysis.

Once validated, the prescriptive analytics model is applied to real-world situations. Decision-makers receive actionable insights and recommendations based on the analysis. For example, in healthcare, prescriptive analytics can assist doctors in determining the most effective treatment plans for patients based on their medical history and current conditions.

Continuous monitoring and review follow the implementation of the recommended actions. This ensures that the prescribed course of action aligns with the desired outcomes and allows for adjustments if necessary. The feedback loop is crucial for refining and improving the prescriptive analytics model over time.

Prescriptive analytics, when executed effectively, guides decision-makers toward the best course of action by considering multiple factors, potential constraints, and desired objectives. It empowers organizations to make data-driven decisions that optimize processes, enhance efficiency, and ultimately lead to successful outcomes.

Applying an appropriate programming language or tool to demonstrate how these prescriptive analytic techniques are used to find the best course of action in a situation.

Using Tableu Public

Tableu public can be an affective tool which can be used in order to demonstrate the prescriptive analytic techniques and create the satisfied dashboards. Tableau empowers users to conduct scenario planning by creating interactive dashboards. Decision-makers can simulate different scenarios and observe the potential outcomes in real-time. For instance, they can explore the impact of adjusting marketing strategies, pricing models, or product offerings. Tableau then prescribes the most favorable scenarios based on the desired business objectives.

Tableau reveals a segment of customers with a high likelihood of responding positively to a new product. Prescriptive analysis would then recommend tailoring marketing messages to resonate with this segment, potentially leading to increased sales. The interactive nature of Tableau dashboards allows decision-makers to explore and refine strategies collaboratively.

In essence, prescriptive analysis in Tableau transforms data into actionable insights, guiding decision-makers to make informed choices that drive positive business outcomes. It's about leveraging data not just for understanding and prediction but also for actively shaping the future direction of the business.

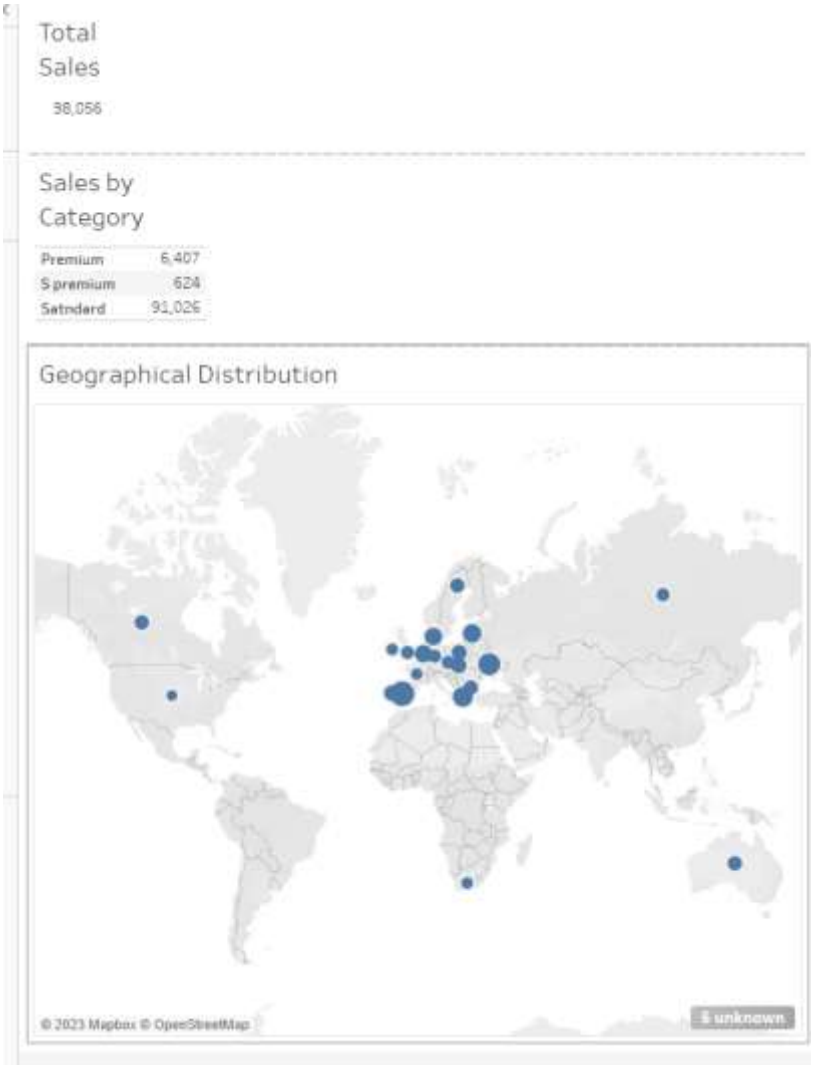
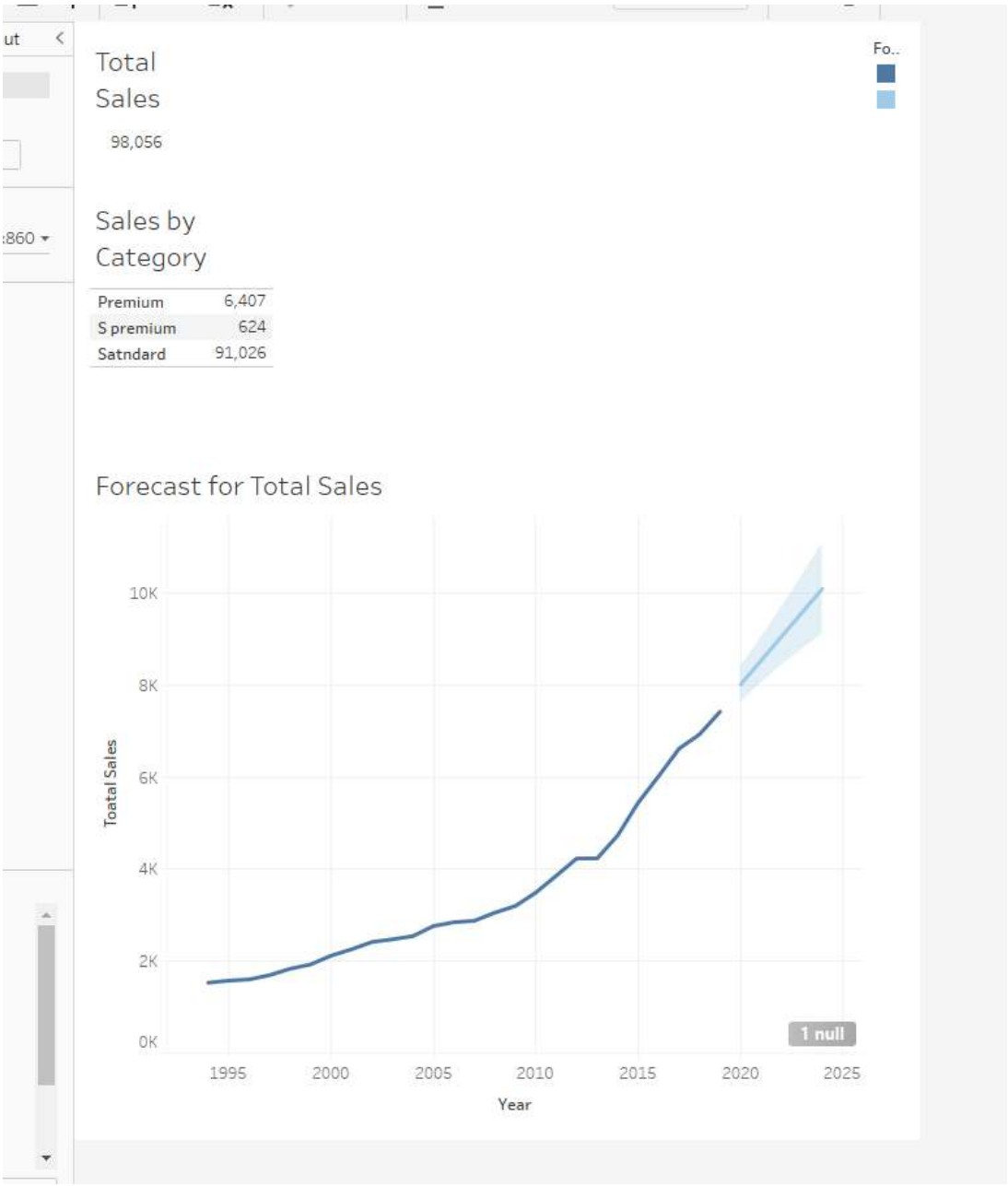
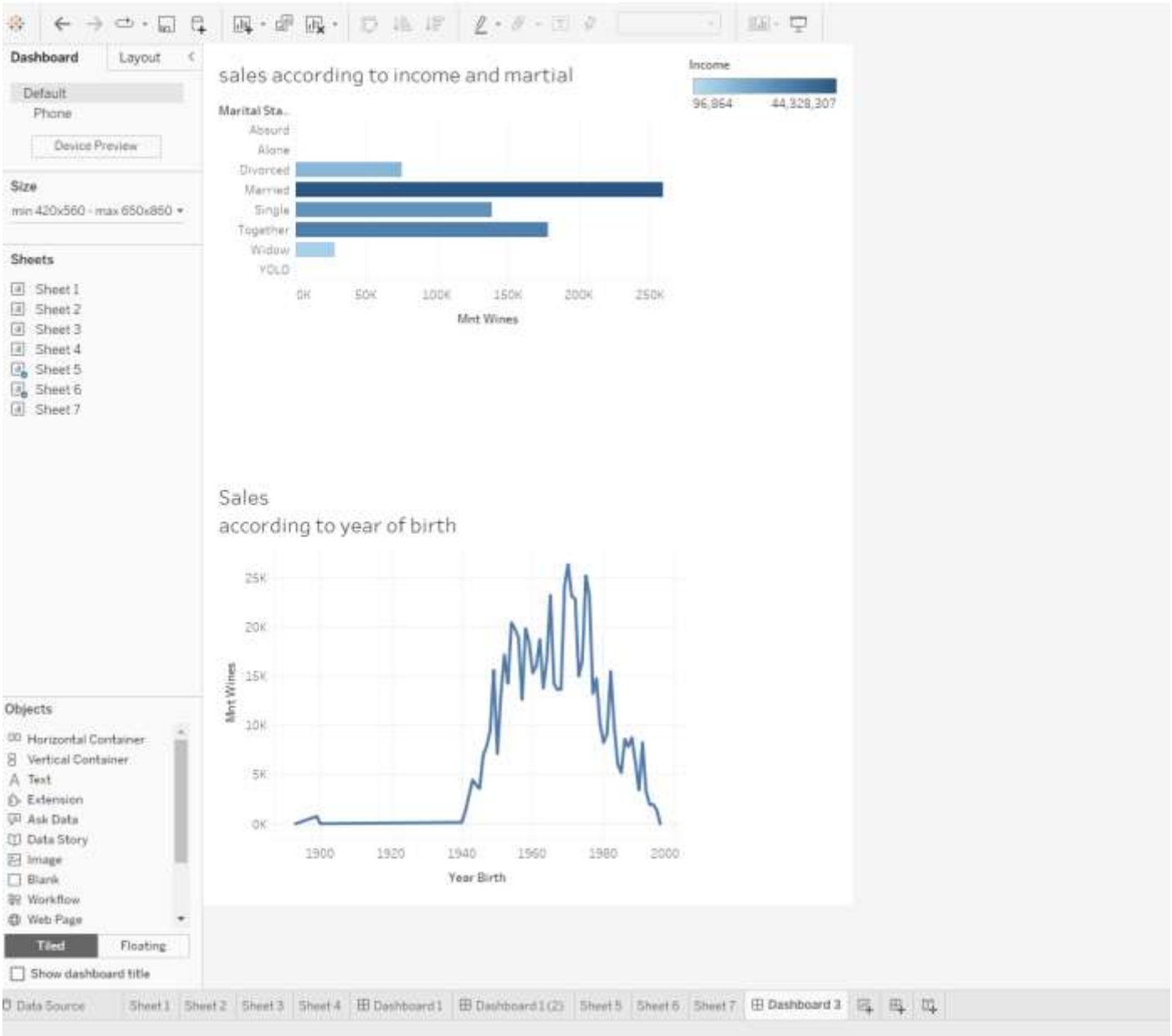


Figure 45 - a dashboard visualizing total sales and geographical distribution

Here in the above dashboard in sales and total sales. It can be identified that a geographical expansion can be considered by the organization. The organization can use these kinds of data for prescriptive analysis and consider what kind of promotion strategies they can utilize in order to increase their goals. Tableau's capabilities extend to optimization, where it can recommend the most efficient allocation of resources. For example, if there's a limited marketing budget, Tableau can guide the optimal distribution of funds across different channels to maximize returns. This prescriptive approach ensures that businesses invest resources where they are likely to have the greatest impact.



Distribution should be considered to be done as it is already done as there is a constant increase during the forecasting. But necessary steps could be taken to increase revenue. Integrating predictive models into Tableau enables the system to forecast future customer behavior. For instance, Tableau can predict the likelihood of customers churning or the probability of making additional purchases. Based on these predictions, prescriptive recommendations could include targeted interventions, such as launching retention campaigns for customers at risk of churning or suggesting cross-selling opportunities.



These dashboards can be utilized in prescriptive analysis so that the organization can consider promotion methods and targets. Tableau allows users to create dynamic visualizations that segment customers based on various criteria like age, income, and past purchasing behavior. By identifying segments that contribute significantly to sales or exhibit unique characteristics, Tableau helps prescribe targeted marketing strategies. For instance, it might recommend personalized promotions for high-value segments to enhance customer satisfaction and loyalty.

Tableau could be employed to identify segments with the highest likelihood of responding positively to a new product launch. The analysis might recommend tailoring marketing campaigns to specifically target these segments, ultimately driving sales growth. Additionally, prescriptive analysis could advise on the optimal distribution of marketing budget across different channels to maximize the overall impact.

In simpler terms, prescriptive analysis in Tableau guides decision-makers by recommending specific actions informed by a deep understanding of the data. It transforms complex insights into practical and actionable steps to improve business outcomes.